Master of Science in Engineering: AI and Machine Learning,
Master of Science in Engineering: Software Engineering
June 2026



Augmented Support Engineer

Challenges and opportunities for Agentic Workflows in Support Engineering

Isak Rulander
Simon Ljung

Faculty of Computing, Blekinge Institute of Technology, 371 79 Karlskrona, Sweden

This thesis is submitted to the Faculty of Computing at Blekinge Institute of Technology in partial fulfillment of the requirements for the degree of Master of Science in Engineering: AI and Machine Learning,
Master of Science in Engineering: Software Engineering. The thesis is equivalent to 20 weeks of full-time studies.

The authors declare that they are the sole authors of this thesis and that they have not used any sources other than those listed in the bibliography and identified as references. They further declare that they have not submitted this thesis to any other institution to obtain a degree.

Both authors contributed equally to all phases of the research, including problem identification, solution design, implementation, data collection, analysis, and writing.

Contact Information:
Authors:
Isak Rulander
E-mail: isru21@student.bth.se

Simon Ljung
E-mail: silj20@student.bth.se

University advisor:
Ricardo Britto
E-mail: ricardo.britto@bth.se
Department of DIPT

Company advisor:
Ruwan Lakmal Silva
E-mail: ruwan.lakmal.silva@ericsson.com
Ericsson

Faculty of Computing
Blekinge Institute of Technology
SE–371 79 Karlskrona, Sweden

Internet : www.bth.se
Phone : +46 455 38 50 00
Fax : +46 455 38 50 57

Abstract

Background. Support engineers in telecommunications spend much of their time navigating fragmented tools and manually analyzing large log files during fault diagnosis. Agentic AI systems, which decompose tasks across multiple tools and iterate through rounds of evidence retrieval, offer a way to reduce this burden. However, no empirical studies have evaluated their impact on practicing support engineers in live telecommunications infrastructure.

Objectives. This thesis designs and evaluates agentic AI solutions for information retrieval in telecommunications support engineering. It measures their effect on engineer performance and cognitive load and their accuracy in retrieving relevant information, under an architecture that enforces read-only access through the Model Context Protocol and keeps all diagnostic decisions with the engineer.

Methods. Following the Design Science Research Process, interviews with support engineers at Ericsson identified the main bottlenecks, and two agentic solutions were built, one for log analysis and one for information search across six internal systems. Each was evaluated in two ways. Six engineers per case completed the same diagnostic tasks once with AI and once without, while accuracy, time, and NASA-TLX cognitive load were recorded. Each agent was also tested on its own: the log analysis agent against 12 trouble reports and the information search agent against a 30-case corpus run three times for 90 total runs.

Results. Across both cases, AI assistance raised median diagnostic accuracy (log analysis from 2.5 to 4.0 out of 6, information search from 4 to 5 out of 6) and lowered median NASA-TLX workload on all five scales, although session time did not decrease. In the AI-assisted condition, most errors in Case 1 (11 of 13) and half in Case 2 (2 of 4) were attributable to engineer-tool interaction rather than AI failure, indicating that effectiveness depends on both agent accuracy and human-AI interaction design. The log analysis agent, on its own, identified 81% of log-derivable findings in 12 trouble reports, failing mainly in evaluative judgment rather than in evidence retrieval. The information search agent produced a reproducible investigative process, but recovered only about half of the evidence references on average across repeated runs on the same case.

Conclusions. AI assistance improved diagnostic accuracy and reduced perceived cognitive load in both cases, although it did not reduce session time. The shortfall relative to the agent's own accuracy was attributable to engineer-tool interaction rather than the AI's analytical capability. Independent agent evaluations showed that the log analysis agent is a strong evidence retriever and a weak evaluative reasoner, while the information search agent is reproducible in the process but variable in evidence recall. The practical impact of agentic AI on support engineering, therefore, depends on interaction design, engineer training, and the constraints that preserve engineer authority over diagnostic decisions as much as on model capability.

Keywords: Support Engineering, Agentic AI, Model Context Protocol, Log Analysis, Information Search.

Sammanfattning

Bakgrund. Supportingenjörer inom telekommunikation lägger mycket tid på att navigera fragmenterade verktyg och manuellt analysera loggfiler vid felsökning. Agentiska AI-system, som bryter ned uppgifter över flera verktyg och itererar genom flera omgångar av evidensinsamling, kan minska denna börda. Ingen empirisk studie har dock utvärderat deras påverkan på praktiserande supportingenjörer i levande telekommunikationsinfrastruktur.
Syfte. Denna uppsats designar och utvärderar agentiska AI-lösningar för informationssökning i teknisk support. Den mäter deras effekt på ingenjörers prestanda, kognitiva belastning och träffsäkerhet, under en arkitektur som genom Model Context Protocol ger endast läsbehörighet och behåller diagnostiska beslut hos ingenjören.
Metod. Enligt Design Science Research Process identifierades flaskhalsarna genom intervjuer med supportingenjörer på Ericsson, och två agentiska lösningar togs fram, en för logganalys och en för informationssökning i sex system. Sex ingenjörer utförde uppgifterna med och utan AI, medan träffsäkerhet, tid och NASA-TLX-belastning mättes. Varje agent testades även på egen hand: logganalysagenten mot tolv felrapporter och informationssökagenten mot ett 30-ärenden-korpus kört tre gånger (90 körningar totalt).
Resultat. I båda fallen höjde AI-stödet medianträffsäkerheten (logganalys från 2,5 till 4,0 av 6, informationssökning från 4 till 5 av 6) och sänkte medianen på alla fem NASA-TLX-skalor, även om sessionstiden inte minskade. Felen i AI-villkoret dominerades av samspelet mellan ingenjör och verktyg snarare än av AI:ns egna fel, såsom felläsning, ofullständig prompt eller utebliven förfrågan. Logganalysagenten identifierade på egen hand 81% av de loggbaserade fynden över tolv felrapporter, med brister främst i utvärderande bedömning. Informationssökagenten levererade ett reproducerbart förlopp men återkallade i snitt endast hälften av evidensreferenserna vid upprepade körningar.
Slutsatser. AI-stödet förbättrade diagnostisk träffsäkerhet och minskade upplevd belastning i båda fallen, men minskade inte tiden. De flesta felsvaren följde korrekt AI-utdata som ingenjörerna felläste eller inte följde upp, så underskottet berodde mer på samspelet mellan ingenjör och verktyg än på AI:ns analytiska förmåga. Logganalysagenten är stark på evidensinsamling men svag på utvärderande bedömning, medan informationssökagenten är reproducerbar i förlopp men varierande i evidensåterkallning. Effekten beror på interaktionsdesign, upplärning av ingenjörer och begränsningar som bevarar ingenjörens auktoritet över diagnostiska beslut, lika mycket som på modellens förmåga.

Nyckelord: Teknisk Support, Agentisk AI, Model Context Protocol, Logganalys, Informationssökning.

Acknowledgments

We would like to express our sincere gratitude to our industrial supervisor at Ericsson, Lakmal Silva, for his continuous guidance and extensive feedback throughout this project. We also thank our university supervisor, Ricardo Britto, for his valuable feedback and direction. Last but not least, thanks to the support engineers at Ericsson for their time during the interviews and experiments, and for welcoming us into their office while we carried out this work.

Contents

Abstract i

Sammanfattning iii

Acknowledgments v

1 Introduction 1
- 1.1 Problem Statement . . . 2
- 1.2 Aim and Objectives . . . 2
- 1.3 Contributions and Limitations . . . 3
- 1.4 Ethical Considerations . . . 3
- 1.5 Outline . . . 4

2 Background 5
- 2.1 Technical Support Engineering . . . 5
- 2.2 Large Language Models . . . 5
- 2.3 Agentic AI Systems . . . 6
- 2.4 Model Context Protocol . . . 6
- 2.5 Root Cause Analysis . . . 6

3 Related Work 7
- 3.1 Limitations from Related Work . . . 10

4 Method 13
- 4.1 Research Questions . . . 13
- 4.2 Research Method Selection . . . 13
  - 4.2.1 Design Science Research Process . . . 14
- 4.3 Research Design . . . 14
  - 4.3.1 Problem Identification . . . 14
  - 4.3.2 Solution Design . . . 15
  - 4.3.3 Evaluation . . . 16
- 4.4 Data Handling and Ethics . . . 25

5 Results and Analysis 27
- 5.1 Agentic Solutions . . . 27
  - 5.1.1 Case 1: Log Analysis Solution . . . 27
  - 5.1.2 Case 2: Information Search Solution . . . 28
  - 5.1.3 Design Decisions . . . 32

5.2 RQ1: Engineer Performance and Cognitive Load . . . 33
5.2.1 Case 1: Log Analysis . . . 33
5.2.2 Case 2: Information Search . . . 39
5.3 RQ2: Agent Accuracy . . . 46
5.3.1 Case 1: Log Analysis Agent . . . 46
5.3.2 Case 2: Information Search Agent . . . 48

6 Discussion **53**
6.1 RQ1: Impact on Performance and Cognitive Load . . . 53
6.1.1 Log Analysis . . . 53
6.1.2 Information Search . . . 54
6.2 RQ2: Agent Accuracy . . . 56
6.2.1 Log Analysis . . . 56
6.2.2 Information Search . . . 58
6.3 Practical Implications . . . 60
6.4 Research Implications . . . 61

7 Threats and Validity **63**
7.1 Reliability . . . 63
7.2 Internal Validity . . . 63
7.3 External Validity . . . 65
7.4 Construct Validity . . . 66

8 Conclusions and Future Work **69**
8.1 Conclusions . . . 69
8.2 Future Work . . . 71

References **73**

A Interview Guide **77**

B Assessment Questions **79**

C Per-Case Stability Data **87**

D Implementation Details **89**

E Result Visualizations **91**

Chapter 1

Introduction

Telecommunications networks provide services that societies depend on, from emergency calls to financial transactions. When such a network is disrupted, specialized support engineers must identify its root cause by searching system logs, product documentation, and historical case records across multiple disconnected tools. As these distributed systems have grown in complexity, so has the volume of diagnostic data associated with a single fault and the time pressure to resolve it.

Arnold et al. [1], in a systematic review of 87 studies on information overload in the workplace, identify the volume, complexity, and fragmentation of information as factors that degrade task performance. For support engineers, this fragmentation manifests as time spent navigating disconnected systems, reformulating queries across tools with different interfaces, and manually piecing together evidence that spans multiple sources. Delayed diagnosis in turn leads to prolonged service disruption for network operators and their end users.

An agentic system can decompose a task into steps, invoke tools to retrieve evidence from different sources, and iterate through multiple rounds of retrieval and analysis before producing a result [2]. Agentic AI systems, therefore, present an opportunity to reduce this retrieval burden. In a support engineering context, this enables an AI system to search fault databases, read log files, and correlate findings between systems on behalf of the engineer. The Model Context Protocol (MCP) provides a standardized interface for such systems to access organizational data sources, with access control mechanisms that allow restricting operations to read-only [3]. In this research, all MCP servers are configured for read-only access as an architectural design choice required by the live telecommunications infrastructure context.

Introducing AI into diagnostic workflows carries risks that must be addressed in the system's design. Feng et al. [4] find that engineers shift from producing output to critically evaluating AI-generated output, creating a verification burden where significant effort is spent validating automated suggestions. Wu et al. [5] find that language models applied to telecommunications fault analysis are misled by plausible but incorrect root-cause candidates, emphasizing the need for workflows that allow the engineer to retain full authority over diagnostic conclusions. These findings indicate that the value of agentic AI in support engineering depends not only on system accuracy but also on how it is integrated into the engineer's workflow. Existing agentic systems for diagnostics have been evaluated on automated benchmarks rather than through empirical validation by practicing engineers on real faults, leaving it open whether they improve or complicate diagnostic work.

This thesis designs and evaluates agentic AI solutions for information retrieval in

support engineering, with decision-making retained by the engineer.

1.1 Problem Statement

The problem statement for this master's thesis has been defined in collaboration with Ericsson, a multinational telecommunications company. Support engineers at Ericsson diagnose faults whose relevant information is scattered across internal tools and multi-megabyte log files. The interviews conducted in the first phase of this thesis confirmed that this scattering, together with the experience needed to interpret it, slows diagnostic work in practice. Agentic AI systems have been proposed for such tasks, but their evaluations rely on automated benchmarks rather than practicing engineers. As a result, the effect of such systems on engineers' accuracy, time, and cognitive load in real diagnostic work has not been empirically measured.

1.2 Aim and Objectives

The aims define the main purpose of the research, while the objectives outline the specific steps needed to answer the research questions. The aims of this thesis are as follows:

AIM1.
Analyze current support engineering workflows and identify bottlenecks that affect diagnostic efficiency and to select representative trouble reports and case scenarios with verified resolutions as evaluation targets.

AIM2.
Design and implement agentic AI solutions that address the identified bottlenecks across different support engineering activities, operating under read-only constraints suitable for live infrastructure.

AIM3.
Empirically evaluate the agentic solutions through controlled experiments with practicing support engineers, assessing their impact on task performance and cognitive load.

The primary objectives of this research are:

OBJ1.
Conduct semi-structured interviews and workflow analysis to identify information retrieval bottlenecks in the current support engineering process. Select scenarios with verified resolutions and define measurement instruments for task completion time, accuracy, and cognitive load (NASA-TLX assessments [6]).

OBJ2.
Design and implement agentic AI solutions targeting the identified bottlenecks. Validate that the solutions operate within defined safety boundaries through architectural constraints and permission configuration.

OBJ3.
Evaluate the agentic solutions against real support scenarios and collect structured feedback from support engineers. Compare AI-assisted and manual conditions in a within-subjects crossover experiment to determine the impact on task completion time, accuracy, and cognitive load.

1.3 Contributions and Limitations

This research contributes the following:

1. An empirical analysis of the information retrieval challenges faced by support engineers in telecommunications, based on semi-structured interviews and workflow analysis, identifying specific bottlenecks that can benefit from agentic AI assistance.

2. Agentic AI solutions that address the identified bottlenecks, operating under strict read-only constraints to ensure operational safety in critical infrastructure contexts.

3. An empirical evaluation of agentic solutions against real support scenarios and through feedback from support engineers, providing insights into the practical benefits, limitations, and adoption considerations of agentic AI in support engineering.

This research has several limitations. Agentic solutions are restricted to information retrieval and diagnostic support and do not perform active configuration changes on live systems, as this would introduce risk to critical infrastructure. The solutions connect to Ericsson's internal tools and are designed to support any support engineering team within the organization, but the evaluation is conducted with a single team, which may limit the generalizability of the findings. The rapidly evolving nature of both Agentic AI systems and the Model Context Protocol means that specific technical implementations may need to be adapted as these technologies mature.

Methodological limitations include the small sample size (six participants per case), the use of a single LLM (Claude Opus 4.6) whose behavior may not generalize to other models, and the single-team evaluation that limits external validity.

1.4 Ethical Considerations

The topic of agentic AI in technical support engineering raises important ethical, societal, and sustainability considerations that require thoughtful analysis.

From an ethical standpoint, AI-assisted diagnostic systems must not compromise the integrity of critical infrastructure. Telecommunications networks support essential services, including emergency communications, and any diagnostic tool operating in this context must be designed with safety as the primary concern. The agentic architecture developed in this research is designed for read-only access through the Model Context Protocol. All MCP server configurations expose only read operations, and no write endpoints are registered. While no formal adversarial boundary testing

was conducted, the architectural constraint means the agent has no mechanism to invoke write operations even if instructed to do so.

The system is designed to support rather than replace professional judgment. By presenting traceable evidence and leaving all decisions with the engineer, the architecture reduces the risk of over-reliance on automated suggestions. This design choice reflects that while automated tools can improve efficiency, engineers should not depend on them excessively, as new fault patterns may emerge that the system has not encountered.

All human participation was voluntary, with written informed consent obtained before each session. The study was conducted in accordance with GDPR, Ericsson's data governance policies, and the Swedish Research Council's guidelines on good research practice. No sensitive personal data was collected, and all participants are referred to by anonymized identifiers.

From a sustainability perspective, more efficient diagnostics can reduce downtime and improve the use of engineering resources, contributing to operational sustainability. Reducing the cognitive strain associated with complex troubleshooting tasks contributes to a more sustainable work environment for specialized support engineers. From an environmental standpoint, running large language models and agentic systems adds to data center power usage. However, this cost may be offset by the energy waste associated with prolonged manual fault resolution, including redundant logging, repeated failed builds, and the compute resources consumed during extended debugging sessions.

1.5 Outline

The rest of this thesis is outlined as follows. Chapter 2 provides essential information to understand the thesis, covering key topics such as technical support engineering, large language models, agentic AI, the Model Context Protocol, and root cause analysis. Next, Chapter 3 reviews prior studies relevant to this research and identifies gaps in the existing literature. In Chapter 4, the chosen research questions and the Design Science Research Process used in this research are introduced and explained. This chapter also outlines the design and execution of the research. Chapter 5 presents the two agentic solutions produced by the design process and the main findings from their evaluation, focusing on the answers to the research questions. This is followed in Chapter 6, where the results are analyzed in depth and compared with findings from previous studies. Chapter 7 assesses the reliability of the research, including internal and external validity. Finally, Chapter 8 summarizes the study and suggests potential directions for further research.

Chapter 2

Background

The purpose of this chapter is to provide the necessary knowledge and understanding of the subject. Each section introduces a central concept to this research.

2.1 Technical Support Engineering

Technical support engineering is a specialized function within technology organizations where experienced engineers handle technical issues that require deep system analysis beyond standard troubleshooting procedures. In the telecommunications industry, this typically involves engineers who analyze complex faults across multi-layered systems, including application software, platform infrastructure, and network components. The diagnostic process requires synthesizing information from system logs, telemetry data, product documentation, historical case records, and communications with multiple stakeholders, each accessed through different tools and interfaces. In the context of this research, the focus is on the information-retrieval and gathering activities these engineers perform and how these activities can be supported by Agentic AI systems.

At Ericsson, support engineers interact with several internal systems during fault diagnosis. A *ticket* is a structured record of a customer-reported fault, tracked in the Customer Support Ticketing System (CSTS). Related defects are documented in the Trouble Reporting Ticketing System (TRTS) and the Development Ticketing System (DTS). Engineers consult the Product Documentation System (PDS) and the Employee Knowledge Sharing System (EKSS) for product information and known solutions. These systems are described further in Section 5.1.2.

2.2 Large Language Models

A large language model (LLM) is a general-purpose AI system trained to understand, process, and generate text by identifying complex statistical patterns within large datasets. These models serve as analytical engines that reason over unstructured information to synthesize data, extract insights, and perform language-based tasks. In this research, LLMs serve as the reasoning backbone of the agentic architecture, enabling interpretation of technical documentation, the analysis of log patterns, and the generation of diagnostic hypotheses. Base models generate responses solely from patterns learned during training, which means they may provide outdated information or produce plausible but factually incorrect outputs, a phenomenon commonly

referred to as hallucination.

2.3 Agentic AI Systems

Agentic AI systems are AI systems that operate with increased autonomy, capable of independently planning, reasoning, and executing multi-step tasks through tool use and iterative reasoning rather than simple prompt-response patterns [7]. Unlike conventional AI assistants that respond to a single query, agentic systems can decompose a complex task into sub-tasks, select and invoke appropriate tools, evaluate intermediate results, and refine their approach [2]. This capability is particularly relevant for technical support, where root cause analysis often requires traversing multiple data sources, correlating information between systems, and building an evidence chain through iterative investigation. In this research, the agentic architecture is designed to assist support engineers by automating the information gathering process while ensuring the engineer is in control of all diagnostic decisions.

2.4 Model Context Protocol

The Model Context Protocol (MCP) is an open standard that enables a standardized, secure connection between AI models and external data sources such as local files, APIs or databases [3]. An *MCP server* exposes one or more *tools* (discrete operations such as a search endpoint or a lookup function) through this standardized interface, providing strict control over which system resources are visible to the model. This standardization is essential for industrial domains because it allows organizations to enforce boundaries on what data and actions the AI system can access. In this research, MCP is used to govern the exposure of diagnostic data to the agentic system while enforcing read-only constraints that prevent the system from modifying live infrastructure. MCP follows a Host-Client-Server architecture. The *Host* is the application that initiates connections (in this research, the Kiro CLI process). The *Client* maintains a connection to one or more Servers on behalf of the Host. Each *Server* exposes tools and resources through the protocol. This separation allows fine-grained control over which capabilities each server provides to the agent.

2.5 Root Cause Analysis

Root cause analysis (RCA) is the systematic process used in engineering to identify the underlying cause of a fault or failure rather than addressing the immediate symptoms. In telecommunications support, RCA involves analyzing system logs, correlating alarms across components, reviewing configuration states, and comparing observed behavior against expected behavior. The process is often complicated by the multi-layered nature of modern systems, where a single observable fault may have its root cause in a different component or layer entirely. Effective RCA requires both domain expertise and access to comprehensive diagnostic data, making it a natural area for AI-assisted information retrieval.

Chapter 3

Related Work

The development of agentic workflows for technical support engineering draws on research from several areas: information retrieval in expert work, AI-assisted diagnostics, agentic system architectures, tool integration protocols, and human-AI collaboration. This chapter reviews the most relevant studies and concludes with a discussion of the limitations and gaps that motivate this thesis.

Pirolli and Card [8] establish the theory of Information Foraging, which models how humans adapt their search strategies in information-rich environments to maximize the value of information gained relative to the cost of obtaining it. The theory predicts that when information is scattered across sources with poor navigational cues, retrieval cost increases and task performance degrades. This framework provides a theoretical explanation for the challenges observed in our interviews, where support engineers spend significant time navigating between disconnected tools to locate diagnostic evidence.

The Naturalistic Decision Making framework [9] describes how experts make decisions in high-stakes, time-pressured environments. Klein [9] finds that experienced practitioners rely on rapid pattern recognition built through years of practice rather than exhaustive analysis. This explains the knowledge distribution problem identified in our interviews: experienced engineers diagnose issues quickly, while newer engineers lack the mental models to do so. An agentic system that aggregates and structures evidence could partially bridge this experience gap.

A systematic review of 87 studies on information overload in the workplace [1] identifies the volume and complexity of information as key factors that degrade task performance. Arnold et al. [1] report that information overload is associated with burnout and reduced job satisfaction. For support engineers who must synthesize logs, documentation, and system states from multiple disconnected sources during a single investigation, these findings directly motivate the need for tools that aggregate information before presenting it to the engineer.

Several recent systems address this challenge by applying LLMs directly to log data. Lee et al. [10] present LogRESP-Agent, a recursive framework that processes diverse log formats without predefined templates and provides human-readable explanations for detected anomalies. The framework allows engineers to move from searching individual data points to reviewing AI-generated explanations. Unlike our approach, LogRESP-Agent focuses on automated anomaly detection rather than interactive diagnostic support, where the engineer drives the investigation.

Xu et al. [11] propose LogSage, an end-to-end framework for detecting and fixing failures in continuous integration pipelines that combines compact log preprocess-

ing with looking up similar past failures to suggest fixes. Deployed at scale within ByteDance, the system achieves over 80% precision while consuming roughly 12–15% of the tokens required by baseline approaches. Although LogSage includes automated repair tools, the authors report that these tools cover 22% of failure cases and successfully resolve approximately 10.6% through automated fixes, with the remainder still requiring human intervention.

Recent work on trace debugging has revealed significant limitations in current models. The TRAIL benchmark [12] evaluates LLM performance on agentic workflow trace reasoning and finds that even the best-performing model achieves only 11% accuracy when required to both categorize errors and locate their source in execution traces. This indicates that structured trace analysis remains a fundamental challenge and motivates our approach of using agent-driven step-by-step analysis with targeted tool calls rather than attempting to process entire log files at once.

The Flow of Action framework [2] uses multiple agents following standard operating procedures for root cause analysis. Pei et al. [2] demonstrate that a multi-agent approach with structured procedures outperforms single-agent approaches for fault identification. Their work validates the multi-agent architecture pattern, which we explored in our controller-based design. However, Flow of Action is evaluated on automated benchmarks rather than with practicing engineers, leaving open the question of whether improved fault identification translates to improved engineer performance.

Lodha et al. [13] present MCP-Diag, a deterministic, protocol-driven architecture for AI-assisted network diagnostics that converts raw diagnostic output into structured formats to reduce incorrect outputs. This work is closely related to our MCP-based architecture but focuses on fixed diagnostic procedures rather than the flexible, engineer-driven investigation workflow that characterizes support engineering.

The AgentMaster framework [14] combines A2A (Agent-to-Agent) and MCP protocols to retrieve and analyze information across multiple formats, demonstrating that standardized communication protocols enable agents to work across different data sources. Our architecture follows a similar principle by connecting multiple MCP servers to a single agent, though we use a central coordinator model rather than direct agent-to-agent communication.

Tool selection in large API libraries has been studied by Patil et al. [15], who introduce Gorilla, a large language model fine-tuned for API invocation across 1,645 APIs. The study demonstrates that LLMs can learn to select and invoke the correct API from a large library, and that adding a search step significantly improves accuracy. This finding is relevant to our architecture, where the agent must select among the tools provided by nine different MCP servers.

An analysis of the MCP landscape by Hou et al [3] examines the protocol's architecture, capabilities, and security threats. Their work identifies that MCP provides access control and capability negotiation as core design features, offering a foundation for governing what system resources are visible to the model. In our work, MCP serves as the governance layer that restricts the agentic system to read-only access across all connected data sources, a necessary constraint within live telecommunications infrastructure.

The MCP-Atlas benchmark [16] evaluates LLM tool-use competency across 1,000 realistic tasks using real MCP servers. While the best-performing model achieves a

62% pass rate, the most common failure mode is models not calling any tools, followed by partial task completion.

A mixed-methods study by Feng et al. [4] investigates how agentic AI reshapes the engineer's role across experience levels. Through interviews, task-based activities, and artifact reviews with 20 professional software engineers, they find that experienced engineers maintain control through detailed delegation, while less experienced engineers oscillate between over-reliance and cautious avoidance. Seniors reported review fatigue as AI-generated output volumes grew, and the authors warn that engineers may accept AI suggestions without meaningful review when the output appears to work. The study describes a shift where the engineer's core responsibility moves from producing output to critically evaluating AI-generated output.

The trust gap in AI-assisted diagnostics is addressed by Wu et al. [5] through TN-AutoRCA, a benchmark and agentic framework for alarm-based root cause analysis in telecommunications. The benchmark comprises 530 fault scenarios constructed from expert-validated knowledge graphs, and the study finds that even advanced LLMs achieve only roughly 59% F1-score when applied directly. The authors propose a self-improvement mechanism in which an agent repeatedly refines a code-based solution using feedback on its mistakes across all failure cases, boosting F1 from 59% to 92%. This reinforces the importance of our design decision to present evidence to the engineer rather than conclusions, allowing independent verification. Their benchmark construction approach, which derives fault scenarios from structured domain knowledge and verifies root causes, also informs our evaluation design.

Ahuja et al. [17] present VIGIL, an edge-extended agentic AI system for enterprise IT support that deploys desktop-resident agents for diagnosis and policy-governed remediation. In a 10-week pilot across 100 endpoints, the system reduced interaction rounds by 39% and achieved at least 4× faster diagnosis compared to human support, while users reported low cognitive workload (NASA-TLX 2.53/7.0) and high trust (4.29/5.0). All state-changing actions are governed by a deterministic policy engine that classifies commands into allow, warn, or deny tiers, with moderate-risk actions requiring explicit user consent. This tiered governance model aligns with our architecture's read-only constraints, in which the agent retrieves and presents evidence, while the engineer makes all diagnostic decisions.

A review of agentic systems in software engineering [7] identifies human-agent collaboration and orchestration as key open challenges. The review notes that existing benchmarks focus on individual tasks and often overlook the collaborative aspects of software engineering. Our within-subjects crossover design, where each engineer serves as their own control across AI-assisted and manual conditions, directly addresses this gap.

Agentic diagnostic reasoning over telecom and datacenter infrastructure [18] demonstrates the application of multi-step agent reasoning to infrastructure fault diagnosis. This work operates in a similar domain to ours but focuses on automated diagnosis rather than augmenting the human engineer's workflow.

To measure subjective workload, Hart and Staveland [6] developed the NASA Task Load Index (NASA-TLX), a multidimensional assessment tool that captures perceived workload across six scales. NASA-TLX is widely used in human factors research and provides a standardized method for comparing cognitive load across experimental conditions. In our evaluation, we use a modified five-scale version admin-

istered immediately after each assessment round to capture the subjective workload difference between AI-assisted and manual analysis.

Taken together, the reviewed works show that the technical components for AI-assisted diagnostics exist. LLMs can process logs, agentic architectures can structure multi-step investigations, and MCP can govern tool access. What is missing is evidence that these components actually help the engineer. Every diagnostic system reviewed above is evaluated on automated benchmarks or synthetic scenarios, and the only study examining the engineer's changing role with agentic AI focuses on software development, not diagnostic support. The following section details the specific gaps that emerge from this observation.

3.1 Limitations from Related Work

Several significant gaps remain that motivate this thesis:

- **Lack of empirical evaluation with practicing engineers.** Existing agentic systems for diagnostics, including Flow of Action [2] and TN-AutoRCA [5], are evaluated on automated benchmarks or synthetic scenarios. We found no study that measures the impact of agentic tools on the actual task performance of support engineers working on real fault cases.

- **Lack of controlled comparison of AI-assisted and manual analysis.** The over-reliance and review fatigue identified by Feng et al. [4] are described qualitatively but not quantified. We found no study with controlled experiments that compares AI-assisted and manual support engineering tasks by measuring both objective performance (accuracy and time) and subjective cognitive load.

- **Lack of MCP-enabled system evaluated in a support engineering context.** While Hou et al. [3] analyze MCP's architecture and security properties, and the MCP-Atlas benchmarks tool-use competency. We found no study that evaluates an MCP-based agentic system used by engineers in a near-production telecommunications environment.

- **Support engineering is underrepresented in the literature.** Information retrieval, expert decision making, and AI-assisted diagnostics are studied independently, but the intersection of these areas within the specific context of telecommunications support engineering remains largely unexplored. The few studies that address the engineer's changing role [4] focus on software development rather than diagnostic support tasks.

- **Limited comparison of agent architectures for diagnostic tasks.** While multi-agent approaches are shown to outperform single-agent approaches in automated benchmarks [2], no study compares these architectures in terms of their practical impact on engineer workflow, including factors such as response time and usability. This thesis provides a preliminary comparison through batch-mode accuracy testing (Section 5.3.2), but does not conduct a full engineer-facing comparison between architectures, which remains future work.

This thesis addresses these gaps by designing agentic AI solutions for support engineering tasks and evaluating them through controlled crossover experiments with practicing support engineers.

Chapter 4

Method

This chapter presents the research questions, the selected research methodology, and the research design used to achieve the aims and objectives of this study. It includes a description of the Design Science Research Process and its application across three phases: problem identification, solution design, and evaluation.

4.1 Research Questions

To achieve the aims and objectives, the following research questions are addressed:

RQ1: What is the impact of the agentic solutions on support engineer performance and cognitive load?
RQ1 evaluates the effect of AI-assisted workflows compared to manual methods. By measuring task completion time, accuracy, and cognitive load (using the NASA Task Load Index [6]), this question establishes whether the solutions deliver measurable benefits for practicing engineers. RQ1 is supported by OBJ1, OBJ2, and OBJ3.

RQ2: How accurate are the agentic solutions in retrieving relevant information for support engineering tasks?
RQ2 investigates whether the agentic solutions can reliably identify and present relevant information across different support engineering activities. Using scenarios with known correct outcomes, this evaluation determines the accuracy of the solutions' information retrieval and reasoning. RQ2 is supported by OBJ2 and OBJ3.

4.2 Research Method Selection

The methodology for this research is the Design Science Research Process (DSRP) as proposed by Offermann et al. [19]. DSRP supports the iterative development and evaluation of artifacts in collaboration between academia and industry. DSRP operates in three phases: problem identification, solution design, and evaluation. This aligns with the objectives of this research, which include identifying bottlenecks in support engineering workflows, developing agentic AI solutions to address them, and evaluating the solutions with practicing engineers.

Alternative methodologies were considered but found less suitable:

- **Case Study:** A case study methodology would allow in-depth investigation of the phenomenon in its real-world context, but it does not inherently include the design and development of an artifact, which is central to this research.

- **Action Research:** Action research iteratively intervenes in an organization and reflects on the resulting change, which fits industry-academia collaboration. Its focus, however, is the organizational change rather than the artifact itself. DSRP keeps the artifact as the deliverable, which matches the two agentic solutions evaluated in this thesis.

4.2.1 Design Science Research Process

The Design Science Research Process consists of three phases:

- **Problem Identification:** The initial phase involves identifying and confirming the importance of the problem through literature review, interviews with practitioners, and workflow analysis.

- **Solution Design:** In this phase, solutions are developed through an iterative engineering process informed by the findings from problem identification and guided by the related work.

- **Evaluation:** The final phase involves evaluating the developed solutions through controlled experiments with practicing engineers.

The process supports non-linear progression where phases may overlap. In this research, findings from a pilot evaluation session led to revisions of the assessment questions and experiment protocol before the main evaluation was conducted.

4.3 Research Design

This section describes the activities undertaken in each phase of the research. The purpose is to provide a clear and reproducible description of the research process. The phases are presented in order: problem identification, solution design, and evaluation.

4.3.1 Problem Identification

In the problem identification phase, Ericsson tasked us with investigating how agentic AI can support engineers' work. To understand the current state, several complementary activities were conducted during this phase to identify bottlenecks in support operations and establish the foundation for the solution design.

Six semi-structured interviews were conducted remotely via video call with Ericsson support engineers, each lasting between 30 and 45 minutes. At least one author was present at all interviews. Participants ranged from engineers with a few years of experience to those with over fifteen years in the role. Each interview followed four phases: triage and information gathering, troubleshooting and problem solving, bottlenecks and efficiency, and tools and future improvements (see Appendix A).

All interviews were recorded with the participant's consent and anonymized in the analysis.

Both authors independently reviewed the recordings and noted recurring themes, then compared notes and discussed disagreements until consensus was reached. The most consistent findings were that search tools are fragmented and difficult to use, that log analysis is manual and time-consuming, that experienced engineers rely heavily on personal knowledge and contacts that newer engineers lack, and that the quality of incoming tickets is often too poor to begin investigation without requesting additional information. Additional themes included concerns about data privacy when using AI tools, distrust of outdated knowledge base articles, and frustration with internal communication tools.

These themes were used to identify candidate areas for agentic AI solutions. Table 4.2 ranks the bottlenecks by how frequently each was identified as a top concern across the six interviews, the ranking reflects mention frequency, not a formal ranking exercise. The selection of cases for solution development is described in the following section.

4.3.2 Solution Design

The interviews and workflow analysis identified several bottlenecks in the support engineering process. Not all of these could be addressed with agentic AI, nor could all be pursued within the timeframe of this thesis. This section describes how the bottlenecks were grouped into candidate cases for agentic support, which of those cases were taken forward for implementation, and the design of each selected solution.

From these bottlenecks, four candidate cases for agentic support were initially considered, as summarized in Table 4.3. Two were taken forward for implementation and evaluation: log analysis and information search. These were selected based on feasibility within the project timeframe, availability of historical cases with verified resolutions for evaluation, and suitability for a read-only agentic architecture. The remaining two, knowledge sharing and customer connectivity, did not meet all three criteria and are revisited in the future work discussion.

Both solutions were developed following the iterative Design Science Research pattern described by Silva [20], with each design iteration motivated by failure modes observed in the preceding one. The log analysis solution was refined through a pilot session with an experienced engineer. The information search solution went through four versions (v1–v4), evaluated with a paired testing harness in which every case is run twice against the same agent, once with the solution fields stripped from the customer support platform (the masked run) and once with the raw case. Pairing each case in this way characterizes the workload reproducibly across versions, as recommended by Blackburn et al. [21]. The iteration corpus comprised 17 support cases drawn at random from cases resolved within the preceding five months, stratified across three priority levels, and each case was re-run against all four versions to enable paired comparison.

The resulting artifacts are described in Chapter 5, Section 5.1, together with the iteration outcomes and design decisions that shaped the final designs.

4.3.3 Evaluation

The evaluation phase assesses the agentic solutions through controlled experiments with practicing support engineers at Ericsson and through separate accuracy tests of the solutions themselves. The engineer experiments address RQ1 by measuring the impact on task performance and cognitive load. The accuracy tests address RQ2 by evaluating the solutions against scenarios with known correct outcomes independently of the engineer. Two cases were evaluated, log analysis and information search. The engineer experiments follow a within-subjects crossover design, where each participant completes tasks both with and without AI assistance, serving as their own control. This section describes the experiment design, participant recruitment, protocols for each case, workload measurement, and agent-accuracy evaluation.

4.3.3.1 Experiment Design

A true experiment requires random assignment of participants drawn from the target population. In this study, the participant pool was restricted to a single support team at Ericsson, and participants were randomly drawn from within that team rather than from the broader population of support engineers. The engineer studies are therefore quasi-experiments [22]: random selection is applied within the team, but the team itself is a convenience frame. Within that constraint, a within-subjects crossover design [22] was selected to control for individual differences in experience and skill level. In this design, each participant completes two rounds, one with AI assistance and one without. Participants are assigned to one of two groups, differing in which round includes AI assistance, while keeping the scenario order fixed (Table 4.4). A two-group design was chosen over a fully counterbalanced four-group design because the small number of available participants (six per case) would result in only one or two participants per group, making the results vulnerable to individual variation [22]. By using two groups with a fixed scenario order, the design ensures that the only difference between groups is when AI assistance is provided, allowing the effect of AI to be compared directly. A consequence is that scenario position is confounded with round position (Scenario 1 is always seen first). Varying them independently would require four groups, which the sample size does not support, and scenario-level reporting in the results is used to disentangle the effects of scenario difficulty from AI-assistance effects.

Each round consisted of six multiple-choice questions that required the engineer to analyze the provided material. In the manual round, engineers used their usual tools. In the AI-assisted round, the agentic tool was available alongside the usual tools. Engineers were free to use whichever combination of tools they preferred.

A custom web-based benchmark application was developed to administer the experiments. The application presents the necessary files for each scenario, guides the participant through the six questions sequentially, and records the time spent on each question, the selected answer, and the total session duration. Participants cannot return to previous questions. This simulates irrevocable triage decisions and prevents revisiting with knowledge gained later in the investigation. Before the measured rounds, participants complete a short practice round to familiarize themselves with the interface.

Before the main evaluation, one experienced engineer piloted the protocol end-to-end. This engineer works with the same product but is not part of the support team from which the six main participants were recruited. The log analysis case was piloted first, followed by the information search case. The pilot sessions served only to validate the protocol, and the data from the pilot runs is not included in the experiment results. Feedback from the pilot runs and the resulting revisions are described in each case's section.

4.3.3.2 Case 1: Log Analysis Experiment

The log analysis experiment evaluates whether the agentic log analysis tool improves the accuracy and speed of fault diagnosis compared to manual analysis. Two historical trouble reports with verified root causes were selected from the Trouble Reporting Ticketing System (TRTS), one per round of the crossover design, and are referred to throughout the thesis as Ticket-A and Ticket-B. Ticket-A involved a subsystem logging excessive exceptions for non-existent subscribers, with a 9 MB binary CSV transaction log and a 6 MB application log. Ticket-B involved a platform upgrade that changed the message-bus memory allocation, triggering a rolling restart that caused message-bus timeout errors overlapping with pre-existing coordination-service connectivity issues, captured in a 15 MB component log and a 3 MB test case log. The cases were chosen because the original log files were still available, the root cause had been identified and documented, and the two cases represent different fault categories and log formats so that a scenario-specific effect cannot be confounded with the AI-assistance effect.

The full question set is reproduced in Appendix B. Questions are scored binary correct/incorrect against a single pre-authored correct option, which was verified against the actual log data during question design to ensure that each correct answer is evidenced by the logs alone.

In the AI-assisted condition, participants use the log analysis tool, which accepts uploaded log files and provides an interactive chat interface for querying them. In the manual condition, participants analyze the log files directly using their usual tools (text editors, the terminal, command-line utilities such as `grep` and `strings`, and spreadsheet tools for format conversion where needed).

The pilot session, in which the pilot engineer completed the same two trouble reports (Ticket-A and Ticket-B) later given to the six main participants, revealed that some multiple-choice questions required system knowledge beyond what the logs contained, and that certain distractors could not be ruled out from the available data. The affected questions were revised before the main experiment to ensure that each correct answer could be verified from the log files alone, and the experiment instructions were clarified. The open-ended evaluation on 12 additional trouble reports (Section 5.3.1) was not part of the pilot and was run only against the agent.

4.3.3.3 Case 2: Information Search Experiment

The information search experiment evaluates whether the agentic information retrieval tool improves the speed and quality of initial case assessment compared to manual search. Two historical customer support tickets were selected from the Cus-

tomer Support Ticketing System (CSTS), one per round of the crossover design, and are referred to throughout the thesis as Ticket-C and Ticket-D. Ticket-C involved a monitoring heartbeat flapping on a product node, with the root cause recorded as a configuration defect on the customer side and a related known issue in a trouble report. Ticket-D involved an asynchronous request that became stuck at the northbound interface, with the root cause recorded as a software defect tracked by a trouble report and a referenced DTS ticket. The two tickets differ in customer, problem category, and recorded resolution, so that a scenario-specific effect cannot be confounded with the AI-assistance effect. Ticket-D was selected as the more complex of the two cases, with a deeper root-cause trace and more cross-system evidence required, so that the pair would span a range of diagnostic difficulty.

Each ticket is already resolved upstream, with a technical answer and the supporting evidence recorded on the CSTS ticket record. To prevent the participant from reading the answer, the benchmark proxies every ticket fetch through a masking layer that strips the resolution-bearing fields, forces the ticket's visible status to "Analysis", and drops feed items and history rows that would reveal the closure. Candidate cases were also pre-screened by the authors against the support team roster so that no recruited participant had been the case owner or a team member on either ticket. This check is manual rather than enforced by the platform.

Each case was assessed through six multiple-choice questions. The full question set is reproduced in Appendix B. Questions are scored binary correct/incorrect against a single pre-authored correct option, which was keyed to the documented resolution during question design. For each question, the benchmark records the selected answer, whether it was skipped, and the time spent on that question. For each session, it records start and end times, total duration, the explicit consent flag, the per-scale NASA-TLX ratings, and an optional feedback text.

In the AI-assisted condition, engineers used the multi-agent architecture (Section 5.1.2), which performs a three-round autonomous investigation across the connected MCP servers before presenting aggregated findings. The engineer initiates the multi-agent investigation by submitting the case identifier, after which the three-round workflow executes autonomously for approximately 10 to 12 minutes before returning a structured synthesis. This time is included in the reported AI session time. Unlike the log analysis case, where a multi-agent approach was replaced by a single agent because the coordination overhead made responses too slow for interactive use, the information search interaction is batch-style: the engineer submits one prompt, receives a complete investigation, and then optionally asks follow-up questions. In the manual condition, engineers used the underlying Ericsson systems through their standard web interfaces: the Customer Support Ticketing System (CSTS), the Trouble Reporting Ticketing System (TRTS), the Development Ticketing System (DTS), the Product Documentation System (PDS), Employee Knowledge Sharing System (EKSS), and local shell tools. These are the same systems that the AI-assisted condition accesses programmatically through MCP servers.

To ensure the agent was attempted before the participant fell back to manual tools, the benchmark gated progress on an explicit "open the AI tool" step and recorded whether this step was taken. Prompts and responses exchanged with the agent are persisted by the Kiro chat store on a mounted volume (Section 5.1.3), and are used in the qualitative analysis of Section 5.2.2. They are not linked to the

benchmark's per-question timings because the agent runs as a separate application in a new browser tab.

As in Case 1, feedback from the pilot was incorporated before the main experiment.

4.3.3.4 Workload Measurement

Immediately after each round, participants completed a modified NASA Task Load Index (NASA-TLX) assessment [6]. The original TLX comprises six scales. This study uses five of them: Mental Demand, Time Pressure, Performance, Effort, and Frustration, excluding Physical Demand because the tasks involve no physical activity. Each scale is rated on the 21-point scale defined in the original instrument, without the pairwise weighting procedure used to derive the composite workload score in the full instrument. The TLX was administered after each round rather than at the end of the session, so that the workload specific to each condition could be captured while the experience was still fresh.

4.3.3.5 Participant Recruitment

Participants were recruited from the support engineering organization at Ericsson using convenience sampling. The target population is support engineers working on a telecommunications product at Ericsson who perform diagnostic and information retrieval tasks as part of their regular work. Participation was voluntary, and participants were informed of the study's purpose and their right to withdraw at any time. Because the team is small, individual details are not reported to protect anonymity. All participants were experienced support engineers with prior product experience. Experience was not collected as an exact figure, participants reported approximate ranges during recruitment. The overall range spans from a few years to over fifteen years. The crossover assignment was not stratified by experience. All results are reported using anonymized identifiers.

4.3.3.6 Agent Accuracy Evaluation

The agent accuracy evaluation measures how accurately the agentic solutions answer diagnostic questions independently, without a human engineer. This evaluation addresses RQ2 separately from the engineer experiment, isolating the solution's retrieval and reasoning capability from the human interaction layer. The two cases use different accuracy evaluations because their agents produce different kinds of output. Case 1 scores open-ended diagnostic analyses against an expert-authored checklist of log-derivable findings, whereas Case 2 compares paired masked and raw investigation reports on a deterministic metric suite.

Case 1, Log Analysis. The log analysis agent was run independently on the same two scenarios (Ticket-A and Ticket-B) and the same six multiple-choice questions used in the engineer experiment. This mode is called *batch mode* because the agent received the customer report, the attached log files, and all six questions in a single prompt. By contrast, participants in the engineer experiment answered the

six questions sequentially through the benchmark interface. The agent was asked to select one answer per question, with no additional context or hints beyond what the engineers received. Each scenario was run 6 times, for a total of 12 runs. Answers were extracted from the agent's free-form response by pattern matching and compared against the verified correct answers.

To evaluate the agent beyond the multiple-choice format, 12 additional trouble reports with verified resolutions were selected from the Ericsson support platform for open-ended analysis. The trouble reports span 12 distinct fault types across a range of categories, including pod lifecycle failures, migration timeouts, thread-safety bugs, error-mapping defects, storage infrastructure problems, and configuration faults. Internal trouble reports were chosen over customer support requests because they consistently include complete log files and verified resolutions, enabling reliable scoring. For each trouble report, the agent received the customer report and the attached log files without question and was asked to produce a complete diagnostic analysis.

The output was scored against a checklist of log-derivable findings, following the nugget-based evaluation framework of Mayfield et al. [23] for long-form generated reports. For each trouble report, one author read the verified TRTS resolution and the attached log files, produced the checklist, and scored the agent's output against it. Findings requiring code-level knowledge (fix instructions, commit references, version numbers) or TRTS metadata (fault codes, category codes) were excluded. This process yielded 107 log-derivable findings across the 12 trouble reports. Scoring is a strict binary per finding. Each finding is marked Found (agent explicitly stated the finding), Missed (agent did not state the finding), or Incorrect (agent stated something contradicting the finding). Each finding is cross-cited to its resolution source, its log-file location, and the agent's response text so that every verdict is independently verifiable.

Case 2, Information Search. The multi-agent information search architecture (`controller`, described in Section 5.1.2) was evaluated on a corpus of 30 historical support cases. Because these are real production cases, each one carries the priority that Ericsson assigned to it at intake. The corpus is stratified into three tiers of ten cases each according to that priority, referred to as *low*, *medium*, and *high* throughout the thesis, and the priority is used as an approximate stratification variable. Priority is assigned by the customer support system based on customer-reported impact and urgency, not diagnostic difficulty directly. The two tickets used in the engineer experiment (Section 4.3.3.3) are included in this corpus but are not partitioned from the remaining cases during reporting. Each batch runs the 30 cases once; the full evaluation comprises three independent batches, so every case is executed three times in total.

Each case is executed as a paired masked and raw run. The masking follows the same principle as SWE-bench [24], where an agent evaluated on historical tickets sees only the information that existed before the case was resolved, so that the answer cannot be read directly from the record. Two things differ in this study. Resolution content is hidden by removing specific fields from the ticket rather than by filtering on time, because the CSTS records do not have a single clean cutoff and resolution content is interleaved with ordinary case content across the same fields. The agent's

output is also scored on how well the masked run agrees with the raw run rather than against a held-out test suite. Both runs receive the same investigation prompt. The masked run adds one instruction that tells the agent to request the masked version of the ticket on all customer-support-platform tool calls, which causes the MCP server to remove the resolution-bearing fields before the agent sees it.

Each masked/raw pair is scored on five deterministic metrics implemented as regular-expression and set-arithmetic procedures, and on three structural checks. All scoring is automated. The metrics and checks are summarized in Table 4.5, and the pipeline is shown in Figure 4.1.

The five metrics are as follows. Source Recovery is the fraction of evidence references in the raw report that also appear in the masked report, adapted from FActScore's atomic-fact comparison [25]. Diagnostic Convergence measures agreement between the two reports on the root-cause category, the trouble-report identifier, and the implicated component. Its category scheme is inspired by the hierarchical differential-diagnosis framework of Lim et al. [26]. Its component-versus-cause separation is inspired by Cui et al. [27]. Actionability is the fraction of eight checklist items fulfilled by the masked report, covering precise commands, documentation references, risk and rollback, workaround versus fix, and escalation. The rubric is drawn from the CLEAR framework [28] and the PlanGenLLMs planning-evaluation survey [29]. Dispatch Efficiency is the fraction of sub-agent dispatches that are not redundant under an agent-plus-keywords dedup key. Redundancy is its complement, the fraction of duplicated dispatches. The two are reported jointly following TRAJECT-Bench [30] and ReasonEval [31]. In this codebase, the two metrics always give the same number, so they count as a single measurement.

The three structural checks are as follows. Convergence is the fraction of dispatch rounds that introduce at least one intent not seen in any earlier round. A run that keeps re-dispatching the same intents scores below 1.0. Evidence Recovery Fidelity, adapted from SeekBench [32], is the fraction of rounds in which the agent's own output indicated weak evidence and the next round changed at least one dispatch intent. It is defined only on runs containing at least one weak-evidence round and reported as not applicable otherwise. Round count is the number of dispatch-bearing rounds in the run, enabling verification of the architecture's three-round contract. Metrics are reported individually and no composite score is computed. Meimandi et al. [33] show that evaluation frameworks which privilege a single dimension while omitting or underweighting others produce a measurement imbalance that misrepresents real-world system value. Reporting each metric separately avoids imposing an unvalidated weighting on the results.

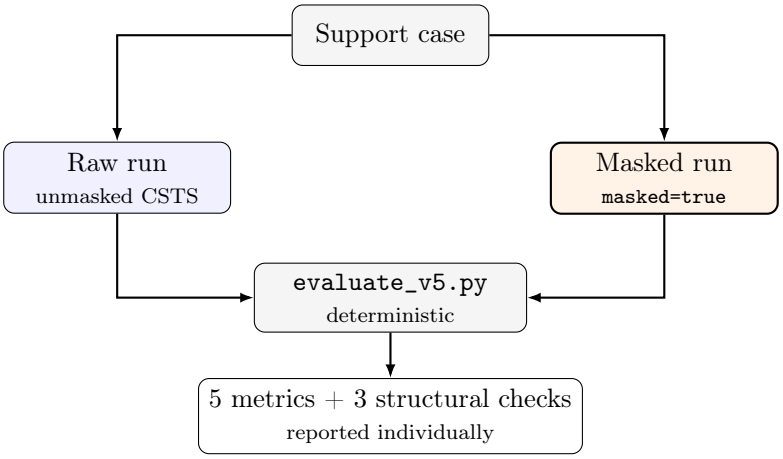


Figure 4.1: Case 2 agent-accuracy evaluation pipeline. Each case is executed twice in parallel: the raw run sees the full CSTS ticket record, the masked run receives the same prompt with instructions that cause the CSTS tool calls to return the ticket with resolution fields stripped. Both reports are then scored deterministically by `evaluate_v5.py`.

Table 4.1: Problem identification activities

Activity	Description
Literature review	An informal review of existing literature on agentic AI, MCP, AIOps, and information retrieval in technical support, conducted by following reference lists and searching for relevant terms. No systematic protocol was applied. The review served as background orientation for the solution design rather than as a standalone research contribution.
Semi-structured interviews	Interviews with support engineers at Ericsson to identify pain points, information retrieval challenges, and requirements for AI-assisted diagnostics.
Workflow analysis	The authors examined the systems used in daily support work and followed engineers as they walked through finished cases, observing how they navigate between tools and what log formats and documentation they consult.
Scenario collection	Gathering representative fault cases through the Ericsson support platform to serve as test scenarios in subsequent phases.
Technical setup	Establishing necessary development environments, access to relevant systems, and MCP infrastructure.
Pre-evaluation of relevance	An early check of whether agentic AI approaches could plausibly address the bottlenecks identified in the interviews, done before solution design began.

Table 4.2: Bottlenecks in support engineering identified from the semi-structured interviews, ordered by participant-assigned priority.

Rank	Identified problem
1	Quality of relevant information
2	Experience and knowledge required to interpret information
3	Difficulty searching for information across tools
4	Customer communication and follow-up
5	Access to the product on the customer side
6	Log analysis
7	Ticket writing (customer cases and trouble reports)
8	Insufficient coverage of test scenarios

Table 4.3: Initial candidate cases for agentic support derived from the interview bottlenecks.

#	Candidate case
1	Information search (“relevance”)
2	Log analysis
3	Knowledge sharing
4	Customer connectivity

Table 4.4: Crossover group assignments. The same design is used for both the log analysis and information search experiments, with scenarios drawn from the two cases in each experiment.

	Round 1		Round 2	
Group	Scenario	Condition	Scenario	Condition
A	1	AI-assisted	2	Manual
B	1	Manual	2	AI-assisted

Table 4.5: Metrics and structural checks produced per masked/raw pair by `evaluate_v5.py`. Scoring is deterministic (regex and set arithmetic); no step is LLM-judged or human-rated. Metrics are reported individually to avoid imposing an unvalidated weighting across dimensions [33]. Dispatch Efficiency and Redundancy are complementary views of the same ratio and coincide numerically on this codebase; both are reported for transparency but one of them can be omitted without information loss.

Metric / check	What it measures	Range	Adapted from
Source Recovery	Recall of evidence references found in the raw report	$[0, 1]$	[25]
Diagnostic Convergence	Agreement on root-cause category, TR, and component	$[0, 1]$	[26, 27]
Actionability	Eight-item checklist on commands, docs, risk, rollback	$[0, 1]$	[28, 29]
Dispatch Efficiency	Share of non-redundant sub-agent dispatches	$[0, 1]$	[30]
Redundancy	Share of duplicated dispatches (complement of Dispatch Efficiency)	$[0, 1]$	[31]
Convergence (check)	Fraction of dispatch rounds introducing a previously-unseen intent	$[0, 1]$	—
Evidence Recovery Fidelity (check)	Fraction of weak-evidence rounds followed by a dispatch-intent change (n/a on runs without weak rounds)	$[0, 1]$	[32]
Round count (check)	Number of dispatch-bearing rounds (architecture contract: 3)	$\mathbb{N}$	—

Two properties of this metric bundle need to be stated explicitly before the results are interpreted. First, Dispatch Efficiency and Redundancy are defined as complementary views of the same ratio and produce numerically identical outputs. They contribute a single independent measurement rather than two. Second, when the raw report produces no parsable evidence references, Source Recovery and Diagnostic Convergence return 1.0 by convention rather than being undefined, and such vacuous scores contribute to the reported means. The Evidence Recovery Fidelity check is informative only on runs that contain at least one weak-evidence round. On other runs, it is reported as not applicable rather than defaulting to PASS.

4.4 Data Handling and Ethics

The research involves four data categories. The first is a series of interview recordings with Ericsson support engineers. The second is experiment session data, including answers, timings, and NASA-TLX ratings. The third is screen recordings and agent interaction transcripts from the AI-assisted rounds. The fourth is the diagnostic material used as experiment scenarios.

Interview and experiment participants were adult professional engineers who volunteered after being informed of the study's purpose and their right to withdraw

at any time. Written informed consent was obtained before each session. Participants are referred to by anonymized identifiers (P1–P6). No names, roles, teams, or other attributes that could re-identify individuals are included in the thesis. Results are reported per participant using these anonymized identifiers. No demographic or organizational attributes are linked to the identifiers.

All data was stored on Ericsson-managed, access-controlled infrastructure, with access restricted to the two thesis authors and the Ericsson supervisor. No participant or diagnostic data left the Ericsson environment during the study. The diagnostic material used as experiment scenarios is Ericsson-internal and is not reproduced in this thesis. Appendix B reproduces only the experiment questions, with product-specific names replaced by generic terms. Interview recordings and raw experiment data will be deleted within six months of thesis submission. Only the anonymized, aggregated data presented in this thesis will be retained.

All AI-assisted interactions were conducted through Kiro, an AI development tool from Amazon Web Services (AWS) approved for use within Ericsson. No internal log data, trouble reports, or case material were submitted to any other AI service during the study. Under the enterprise agreement governing Kiro's use within Ericsson, submitted data is not used for model training.

The study was conducted in line with the General Data Protection Regulation [34], Ericsson's internal data governance policies, and the Swedish Research Council's guidelines on good research practice [35]. The study does not collect sensitive personal data as defined in GDPR Article 9 (e.g., health, political views, or biometrics), nor does it involve physical or psychological intervention in research subjects. It therefore falls outside the scope of the Swedish Ethical Review Act [36] for mandatory ethical review. The study design was reviewed with the Ericsson supervisor before data collection began.

Chapter 5

Results and Analysis

This chapter presents the agentic solutions produced by the Design Science Research Process described in Chapter 4, followed by the findings from their evaluation. Section 5.1 describes the two solutions, including their architecture, workflow, iteration outcomes, and implementation decisions. Section 5.2 addresses RQ1 (engineer performance and cognitive load) and Section 5.3 addresses RQ2 (agent accuracy).

5.1 Agentic Solutions

The Design Science Research Process yielded two agentic solutions, each targeting one of the selected bottlenecks. Case 1 addresses log analysis through a single-agent architecture with direct shell access. Case 2 addresses information search through a multi-agent architecture with a three-round structured investigation workflow and an optional fourth round for any remaining leads. Both solutions operate under read-only constraints and retain all diagnostic decision authority with the engineer.

5.1.1 Case 1: Log Analysis Solution

The log analysis solution provides support engineers with an interactive tool for analyzing log files through a chat interface. The engineer uploads one or more log files and asks questions in natural language. The system responds with findings, citing specific file names, line numbers, and log entries as evidence.

The solution uses a single agent that reads the uploaded files directly using shell commands such as `grep`, `head`, `tail`, `wc`, `strings`, and `zgrep`. The agent works across log formats. The system prompt instructs it to inspect each file's structure before querying, and to fall back to alternative commands when a file resists standard `grep` inspection (for example, invoking `strings` on a binary file). The agent follows a structured strategy. It first surveys the available files, then reads the beginning and end of the most relevant file, then performs targeted searches, and typically responds after two to three rounds of tool use. The system prompt enforces three behaviors:

- every claim must cite a file name and line number;
- the agent describes symptoms without speculating beyond what the logs show;
- multiple commands are combined per tool call to keep the tool responsive during interactive use.

A multi-agent setup with specialized agents coordinated by a controller was explored early in development but abandoned for log analysis, as the additional coordination added too much response time for interactive use. A single agent with direct access to shell tools proved faster and produced comparable results for this case. A pilot session with an experienced engineer led to two refinements. The prompt was adjusted to survey supplementary files rather than focus only on files whose names matched the reported symptom, and the web frontend was adjusted to allow uploading multiple files at once.

5.1.2 Case 2: Information Search Solution

The information search solution is an AI assistant that gathers evidence for a support case across the internal systems that support engineers ordinarily consult by hand. Given a case reference or a problem description, it correlates findings from those sources and returns a structured investigation report with inline citations.

The solution connects to six internal systems exposed through nine Model Context Protocol (MCP) servers (Table 5.1). Where two servers serve the same system, the Ericsson-internal server is written with its plain name (e.g., `@PDS`) and the project-developed variant is prefixed with `dev-` (e.g., `@dev-PDS`). All connections are read-only and are enforced by the MCP server configurations. An eighth candidate source, a build-status log system, was considered during early prototyping but dropped because it was not referenced in any of the investigation iterations (v1–v4) and added no signal to the evaluation corpus.

Table 5.1: Data sources connected to the information search agent. Each system is exposed as a read-only MCP server.

System	Content used by the agent
Customer Support Ticketing System (CSTS)	Customer case details, chatter, files, related cases
Trouble Reporting Ticketing System (TRTS)	Trouble reports (TRs): root causes, corrections, workarounds
Development Ticketing System (DTS)	Defect tracking, linked issues, fix versions
Repository Hosting System (RHS)	Source code commits and diffs
Product Documentation System (PDS)	Product documentation and configuration guides
Employee Knowledge Sharing System (EKSS)	Knowledge articles, known issues

Two architectures were implemented and evaluated. The *solo agent* is a single agent with direct access to all MCP servers. It issues at least 4 searches per round across different sources and continues iterating until the stop criteria below are met. The *multi-agent* architecture uses a controller that dispatches three sub-agent types

in parallel, each scoped to a coherent subset of the servers (Figure 5.1). The controller dispatches exactly four sub-agents per round. Since there are three types, one is dispatched twice with distinct queries.

Multi-agent LLM systems, in which specialized agents collaborate on subtasks of a larger pipeline, have been applied across diverse domains, such as knowledge graph enrichment from scientific literature [37]. In this project, the multi-agent architecture was pursued alongside the solo agent for four concrete reasons:

1. **Parallel investigation.** The controller dispatches sub-agents concurrently within a round, so independent lines of inquiry across the CSTS, knowledge, and dev-tools agents make progress in parallel rather than sequentially.

2. **Scoped context.** Each sub-agent's context window is limited to a coherent subset of the nine MCP servers, keeping any single agent's prompt, tool schema, and intermediate results small enough to reason over without saturating the context.

3. **Explicit dispatch budget.** Exactly four sub-agent invocations per round is easier to enforce and audit than a free-form sequence of tool calls from a single agent.

4. **A structurally different comparator for RQ2.** Retaining the multi-agent design in parallel with the solo agent provides a second data point for RQ2. The two architectures differ in orchestration, context window scope, parallelism, and enforced search count per round. The batch-mode comparison (Table 5.21) provides initial evidence but does not isolate orchestration as a single variable.

Both architectures follow the same three-round structured workflow, summarized in Figure 5.2, with an optional fourth round that fires only if leads from prior rounds remain unfollowed. Round 1 is discovery and broad search: the case is retrieved from CSTS, and the other sources are queried for the product, component, and error signatures. Round 2 pursues targeted leads (TR IDs, DTS keys, RHS changes, error strings to cross-reference). Round 3 is a *challenge round* in which the agent actively seeks evidence that contradicts its current top hypothesis. For example, if the hypothesis is a product defect, the agent searches CSTS for resolved cases with matching symptoms that turned out to be configuration or operational issues. Round 4 follows any remaining unfollowed leads. The agent stops only when the root cause is supported by at least two independent sources, the challenge round has been executed, a workaround has been found or confirmed absent, and no leads remain.

Each claim in the final report must cite its source, including CSTS case IDs, TR IDs, DTS keys, and document titles, so the engineer can verify each finding.

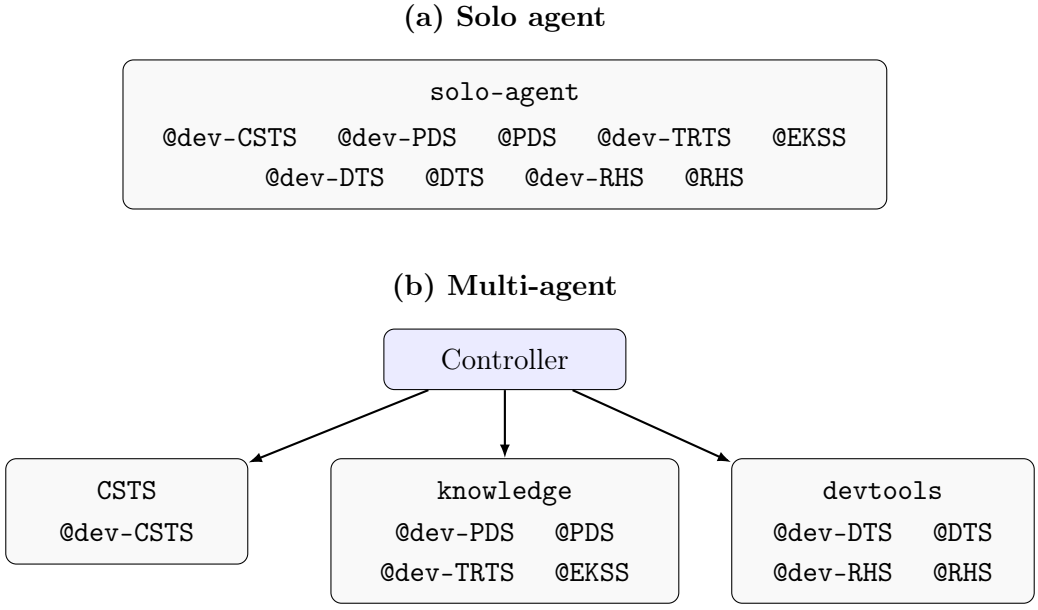


Figure 5.1: The two architectures implemented and evaluated in Case 2. (a) The *solo agent* has direct access to all nine Model Context Protocol (MCP) servers. (b) The *multi-agent* architecture scopes the same nine servers across three sub-agent types. The controller dispatches exactly four sub-agent invocations per round, so one type is dispatched twice with distinct queries. Both architectures connect to the same underlying data sources, isolating orchestration as the independent variable in the evaluation.

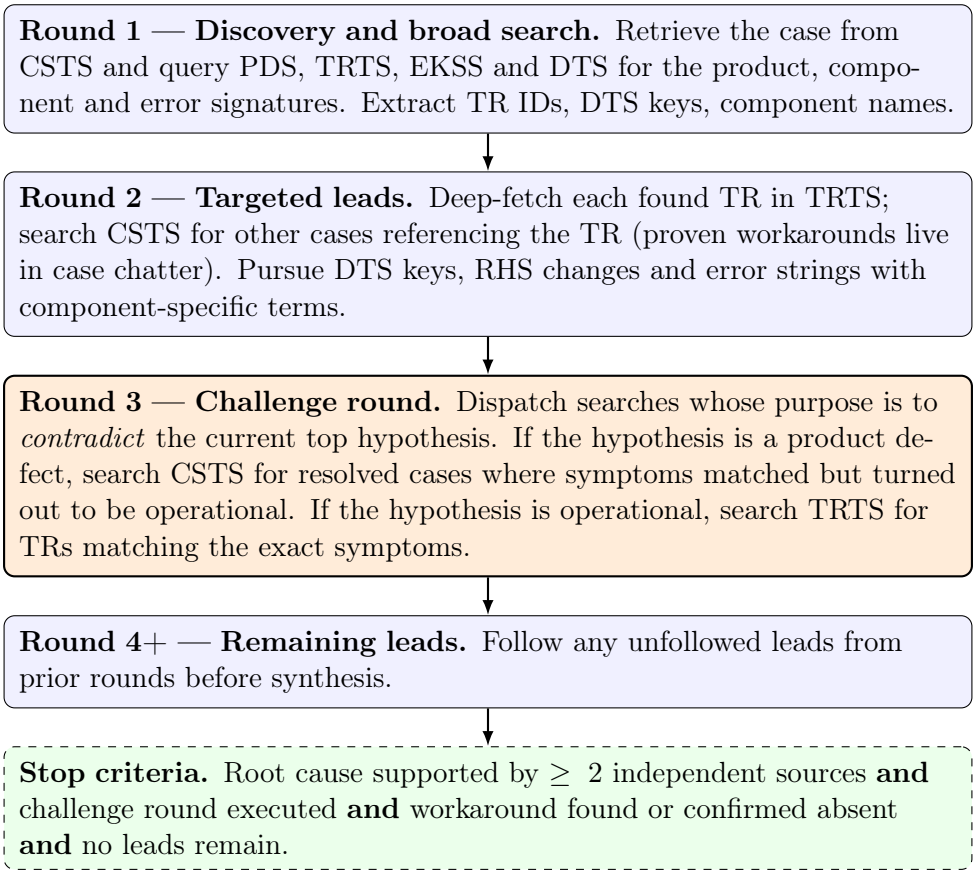


Figure 5.2: Three-round structured workflow (Rounds 1–3) shared by the solo and multi-agent architectures in Case 2, with an optional fourth round to follow up on remaining leads before the stop criteria are checked. Round 3, the challenge round, is the structural counter-measure to confirmation bias introduced in v4; the multi-agent controller dispatches four sub-agents per round (the solo agent performs at least four searches per round).

5.1.2.1 Iteration outcomes

The iteration methodology is described in Section 4.3.2. This subsection reports the outcomes of each version and the design changes that produced the final v4 used in the evaluations reported in Sections 5.2 and 5.3.

v1 used five sub-agents (one per system, eleven MCP servers) as a conservative baseline. v2 consolidated to three sub-agent types, dropped unused build-status tools, and added hypothesis ranking, but regressed on workaround quality. v3 added targeted prompt instructions to improve workaround quality, but the longer prompt led to confirmation bias in complex cases, with repeated runs producing widely different scores depending on which hypothesis was selected first [38].

v4 treated the regressions as premature commitment rather than as a lack of information. The hypothesis ranking and the explicit operational-search instructions were removed, and a *challenge round* was introduced as Round 3, in which the agent dispatches searches whose explicit purpose is to find evidence contradicting its current top hypothesis. Building in a deliberate disconfirmation step is the standard prescription for mitigating confirmation bias in knowledge-intensive tasks [38]. The targeted fixes that had worked in v3 were retained, and the controller prompt was shortened from 6.9 kB to 3.5 kB. v4 is the final design used in the evaluation.

Table 5.2: Iteration of the information search architecture. Each version's changes were motivated by failure modes observed in the preceding version's evaluation.

Version	Structural change	Motivation
v1	5 sub-agents, 11 MCP servers	Baseline
v2	Consolidated to 3 sub-agent types; dropped build-status log system; added hypothesis ranking	Unused tools; redundant dispatches; operational causes missed
v3	Targeted prompt fixes (TR-workaround search, deep-fetch, 3-round minimum)	Workaround quality regression on v2
v4	Added *challenge round*; removed prescriptive ranking; shortened prompts	Confirmation bias on complex v3 cases

5.1.3 Design Decisions

The solutions are built on Kiro, an AI development tool provided by Amazon Web Services (AWS) and approved for use within Ericsson. The agents ran on Kiro CLI version 1.27.2, configured to use Anthropic's Claude Opus 4.6 as the default language model. Claude Opus 4.6 was selected for its strong performance on code and

structured-reasoning tasks and its availability within Ericsson's approved AI tooling through the enterprise Kiro agreement. Model updates released after April 2026 were not evaluated in this study. The implementation architecture, including the Flask API, Next.js frontends, and deployment topology, is described in Appendix D.

Table 5.3 separates the pre-existing components from the authors' contributions. The three-round structured workflow (Discovery, Targeted Leads, Challenge) is specified through the controller's system prompt rather than enforced by deterministic code. The stop criteria are likewise prompt-specified conditions that the LLM self-evaluates. The orchestration logic implemented in code is limited to spawning subagent subprocesses in parallel and passing aggregated outputs to the next dispatch prompt.

Table 5.3: Separation of pre-existing components from authored contributions.

Component	Provider	Authors' contribution
LLM reasoning	Claude Opus 4.6 (Anthropic, via AWS)	Model selection
Tool-calling framework	Kiro CLI (AWS)	Configuration
MCP servers: @PDS, @DTS, @EKSS, @RHS	Pre-existing at Ericsson	SSE transport wrappers
MCP servers: @dev-CSTS, @dev-TRTS, @dev-PDS, @dev-DTS, @dev-RHS	Authors	Full implementation (REST/scraping, auth, tool definitions)
System prompts (both cases)	Authors	All agent and orchestration prompts
Stop criteria and round structure	Authors	Prompt-specified (not code-enforced)
Flask API wrapper	Authors	Full implementation
Next.js frontends (2)	Authors	Full implementation
Scoring harness (30-case corpus)	Authors	Full implementation

5.2 RQ1: Engineer Performance and Cognitive Load

5.2.1 Case 1: Log Analysis

Six support engineers from Ericsson participated in the log analysis experiment in April 2026. Participants ranged from engineers with a few years of support experience to those with over fifteen years in the role. All six participants analyzed Ticket-A and Ticket-B (described in Section 4.3.3.2) in the same order, with the crossover

assignment varying only in which round was AI-assisted. Three participants were assigned to Group A (Ticket-A with AI, Ticket-B without AI) and three to Group B (Ticket-A without AI, Ticket-B with AI).

On aggregate, the AI-assisted condition was associated with a median accuracy of 4.0 out of 6 (compared to 2.5 without AI), longer session time (+4:17 median), and reductions in all five NASA-TLX subjective workload scales at the median level. Individual results varied substantially, with large differences between participants described in the sections below.

5.2.1.1 Accuracy

Table 5.4 shows the number of correctly answered questions per participant and condition. Each participant answered six questions per round. The without-AI column serves as the baseline.

Table 5.4: Log analysis accuracy (correct out of 6) per participant and condition. Participants are sorted by group. The Delta column shows the change when AI was available.

ID	Group	Scenario with AI	Without AI	With AI	Delta
P1	A	Ticket-A	2	4	+2
P3	A	Ticket-A	3	5	+2
P4	A	Ticket-A	0	4	+4
P2	B	Ticket-B	5	5	0
P5	B	Ticket-B	4	2	−2
P6	B	Ticket-B	2	3	+1
Median			2.5	4	+1.5

Four participants scored higher with AI (+1 to +4), one maintained their score (0), and one (P5) scored lower (−2). The median score changed from 2.5 to 4.

Table 5.5 breaks down accuracy by scenario to separate the effect of AI from the effect of scenario difficulty. Because the crossover design assigns each scenario to both conditions, the same scenario can be compared across groups.

Table 5.5: Log analysis accuracy by scenario and condition, showing how the same scenario was scored under opposite conditions by different groups.

Scenario	Condition	Participants	Scores
Ticket-A	Without AI	P2, P5, P6 (Group B)	5, 4, 2
Ticket-A	With AI	P1, P3, P4 (Group A)	4, 5, 4
Ticket-B	Without AI	P1, P3, P4 (Group A)	2, 3, 0
Ticket-B	With AI	P2, P5, P6 (Group B)	5, 2, 3

Ticket-B was harder than Ticket-A under all conditions. Without AI, participants scored 0-3 on Ticket-B, compared with 2-5 on Ticket-A. AI assistance helped on

Ticket-B (scores of 2, 3, and 5), but did not bring all participants to the level achieved on the easier scenario.

5.2.1.2 Time

Table 5.6 shows the session duration per participant and condition. The Delta column shows the change with AI available.

Table 5.6: Log analysis session time per participant and condition. Times marked with * have been corrected for technical disruptions as described below the table.

ID	Group	Without AI	With AI	Delta
P1	A	17:07	21:08*	+4:01
P2	B	51:49	38:22	−13:27
P3	A	33:21	46:40	+13:19
P4	A	27:16*	24:44	−2:32
P5	B	20:19	26:22	+6:03
P6	B	18:55	29:47	+10:52
Median		23:48	28:05	+4:17

Time corrections applied:
*P1 With AI: corrected for 8.5 minutes of tool downtime (raw: 29:38).
*P4 Without AI: corrected for 11 minutes 26 seconds spent downloading keyboard settings during remote session (raw: 38:42).

Wait time during the two technical disruptions was removed from the raw session durations as documented above. The corrected times, therefore, reflect only the participants' active investigation. The +4:17 median increase should be interpreted with caution. The sample is small (n=6), and each participant's two rounds used different trouble reports, so any difficulty difference between Ticket-A and Ticket-B is mixed into their AI-vs-manual time delta. Within these limits, AI assistance did not reduce investigation time in this cohort.

5.2.1.3 Per-Question Analysis

AI assistance had the largest effect on Q2 and Q3, where all six AI-assisted participants answered correctly compared to three of six without AI. These questions required identifying timelines and current system status from large log files, tasks where the AI's targeted search capability provided an advantage over manual scrolling. Q4 was the only question in which the without-AI condition outperformed the AI condition (2 vs 1), and it remained the hardest overall, with only 3 correct answers out of 12 attempts. Q6 (final assessment) showed no difference between conditions. Pairs requiring targeted extraction from large log files (Q2, Q3) benefited most. Pairs requiring final judgment or vocabulary mapping between the AI's output and the answer options (Q4, Q6) did not.

Table 5.7: Log analysis correct answers per question across all participants (n=6 per condition). Each row pairs one question from Ticket-A with one from Ticket-B at the same diagnostic stage. Questions are ordered from initial identification to final assessment.

Q	Ticket-A	Ticket-B	Without AI (of 6)	With AI (of 6)	Δ
Q1	What pattern do failures follow?	What caused the disruption?	3	4	+1
Q2	Do exceptions affect successful requests?	Did the two error types start simultaneously?	3	6	+3
Q3	What error does the client get?	Has the system recovered?	3	6	+3
Q4	What causes the log volume?	What data failed to reach the bus?	2	1	−1
Q5	How many subscribers are failing?	Log volume vs disruption duration?	3	4	+1
Q6	What should happen next?	What should happen next?	2	2	0
Total			16/36	23/36	+7

5.2.1.4 Cognitive Load

Two participants reported large reductions in perceived workload with AI. P6 reported the largest drop across four of the five scales (Mental demand 21 to 5, Time Pressure 18 to 5, Effort 21 to 7, Frustration 15 to 5) on the Performance scale, P4 showed the largest improvement. P5 showed a similar pattern, with mental demand dropping from 14 to 2 and effort from 12 to 6.

P6's workload drop was not matched by an increase in accuracy. Table 5.9 shows P6 sent eight prompts in a mixed pattern, with several off-target questions, and scored 3 out of 6. The drop, therefore, reflects reduced perceived effort without a proportional performance improvement. This is the *false ease* pattern discussed in Chapter 6.

On the Performance scale, which measures self-rated dissatisfaction with one's own performance (lower is better), three participants (P1, P3, P5) rated themselves worse in the AI condition. Two of these (P1 and P3) actually scored higher than without AI, while P5 in fact scored worse with AI despite reporting substantially lower mental demand. Time pressure decreased for one participant (P6, −13), remained unchanged for three (P2, P3, and P5), and increased for two (P1 +4, P4

Table 5.8: Log analysis NASA-TLX ratings (1–21 scale). For all scales, lower values indicate less workload. The Performance scale measures self-rated dissatisfaction with own performance: a lower score means the participant felt they performed better.

		Mental	Time Pr.	Perf.	Effort	Frust.
P1 Without AI	(Ticket-B)	11	11	8	11	8
P1 With AI	(Ticket-A)	10	15	10	12	15
P1 Delta		−1	+4	+2	+1	+7
P2 Without AI	(Ticket-A)	10	10	14	13	14
P2 With AI	(Ticket-B)	10	10	10	13	13
P2 Delta		0	0	−4	0	−1
P3 Without AI	(Ticket-B)	7	5	6	10	7
P3 With AI	(Ticket-A)	5	5	10	10	3
P3 Delta		−2	0	+4	0	−4
P4 Without AI	(Ticket-B)	18	7	13	10	10
P4 With AI	(Ticket-A)	17	10	8	13	12
P4 Delta		−1	+3	−5	+3	+2
P5 Without AI	(Ticket-A)	14	2	12	12	2
P5 With AI	(Ticket-B)	2	2	15	6	2
P5 Delta		−12	0	+3	−6	0
P6 Without AI	(Ticket-A)	21	18	14	21	15
P6 With AI	(Ticket-B)	5	5	10	7	5
P6 Delta		−16	−13	−4	−14	−10

+3), showing no consistent direction at the individual level.

P1 reported higher frustration with AI (8 to 15), influenced by 8.5 minutes of tool downtime during the AI session. P3 reported low frustration in the AI condition (3), the second-lowest across all 12 sessions, after P5, who rated frustration at 2 in both conditions.

5.2.1.5 Qualitative Observations

Six distinct strategies for handling the binary CSV file were observed. The CSV contained embedded null bytes that caused grep to treat it as a binary file and silently suppress output. Strategies included grep with no usable output, the strings command with text file export, AI-assisted Excel conversion, Notepad++ search and filter with visual estimation, AI-assisted CSV parsing, and visual scrolling in a text viewer.

Table 5.9 summarizes how each participant used the AI tool during their AI-assisted round.

Analysis of the 13 incorrect answers across all AI-assisted rounds revealed five error categories, detailed in Table 5.10. Engineers interacted with the AI via a free-form chat interface, so the multiple-choice options were never presented to it. The

Table 5.9: AI usage patterns during AI-assisted rounds. Prompts refer to the number of messages the participant sent to the AI tool. Note that P3, P5, and P6 all sent eight prompts but achieved different scores, illustrating that prompt quantity alone does not determine outcome.

ID	Prompts	Usage style	Score
P1	3	Data extraction only (counts, subscriber totals)	4/6
P4	4	File conversion then targeted questions	4/6
P3	8	Systematic walk-through of diagnostic stages	5/6
P5	8	Broad questions, accepted answers without follow-up	2/6
P6	8	Mixed: some targeted, some wrong questions asked	3/6
P2	10	Interactive investigation with progressively specific prompts	5/6

engineer had to interpret the AI's free-form analysis and map it to the answer options independently.

In total, 8 of 13 incorrect answers occurred when the AI provided correct information that the engineer either misread, selected the wrong option, or drew the wrong conclusion from. In one case, the engineer's prompting led the AI to an incomplete response. In two cases, the engineer did not consult the AI on the topic. The remaining two wrong answers involved AI failures. One was a factually incorrect analysis (P5 Ticket-B Q1), and the other was contradictory responses across turns, with the engineer following the misleading one (P6 Ticket-B Q6). Across all six AI-assisted rounds (36 answers total), 23 were correct, and 13 were wrong, giving an overall accuracy of 64% with AI assistance, compared with 16/36 (44%) in the without-AI condition.

Three of three Group B participants who attempted Ticket-B Q4 with AI made the same error. The question asked: "Data failed to reach the message bus during the disruption. What was it?" The correct answer was option A: "Audit logs of completed operations and alarm clear notifications." All three selected option B: "Incoming provisioning requests and alarm raise notifications." In each case, the AI described "fault indications could not be published" and "processing logs could not be stored." The AI's terminology was technically correct but used different words from the answer options, so engineers had to translate between the two before selecting an answer. The miss was therefore not a retrieval failure but a mismatch between the AI's wording and the answer options.

In summary, AI assistance was associated with higher accuracy for most participants, a shift in errors from retrieval to interpretation, and reduced perceived cognitive load, though the relationship between perceived ease and actual performance was not consistent across participants.

Table 5.10: Classification of all 13 wrong answers in AI-assisted rounds. Each row is one wrong answer.

Participant	Question	Error category
P2	Ticket-B Q4	AI analysis correct, engineer selected wrong option
P5	Ticket-B Q4	AI analysis correct, engineer selected wrong option
P5	Ticket-B Q5	AI analysis correct, engineer selected wrong option
P6	Ticket-B Q4	AI analysis correct, engineer selected wrong option
P6	Ticket-B Q5	AI analysis correct, engineer selected wrong option
P3	Ticket-A Q4	Engineer drew wrong conclusion from correct AI output (AI gave deep and shallow answers; engineer committed to shallower)
P4	Ticket-A Q1	Engineer prompted poorly (incomplete file upload)
P6	Ticket-B Q6	AI gave contradictory responses across turns; engineer followed the later, misleading one
P1	Ticket-A Q4	Engineer did not ask AI about this topic
P1	Ticket-A Q6	Engineer did not ask AI about this topic
P4	Ticket-A Q6	Engineer drew wrong conclusion from correct AI output
P5	Ticket-B Q6	Engineer drew wrong conclusion from correct AI output
P5	Ticket-B Q1	AI provided factually incorrect analysis
Total: 5 wrong option selected, 3 wrong conclusion, 2 not asked, 2 AI error or inconsistency, 1 poor prompting		

5.2.2 Case 2: Information Search

Six support engineers from Ericsson participated in the information search experiment in April 2026. As in Case 1, all six participants analyzed Ticket-C and Ticket-D (described in Section 4.3.3.3) in the same order, with the crossover assignment varying only in which round was AI-assisted. Three participants were assigned to Group A (Ticket-C with AI, Ticket-D without AI) and three to Group B (Ticket-C without AI, Ticket-D with AI).[1]

On aggregate, the AI-assisted condition was associated with a median accuracy of 5 out of 6 (compared to 4 without AI), longer session time (+6:43 median), and reductions in all five NASA-TLX subjective workload scales at the median level. Individual results varied. Five of six participants scored higher with AI (+1 to +2), and one (P1) scored lower (−1). These patterns are discussed in the sections below.

5.2.2.1 Accuracy

Table 5.11 shows the number of correctly answered questions per participant and condition. Each participant answered six questions per round. The without-AI column serves as the baseline.

[1]Participant pseudonyms in this section are local to Case 2 and do not correspond to the same individuals as in Section 5.2.1.

Table 5.11: Information search accuracy (correct out of 6) per participant and condition. Participants are sorted by group. The Delta column shows the change when AI was available.

ID	Group	Scenario with AI	Without AI	With AI	Delta
P1	A	Ticket-C	6	5	−1
P2	A	Ticket-C	4	5	+1
P3	A	Ticket-C	4	6	+2
P4	B	Ticket-D	3	5	+2
P5	B	Ticket-D	4	6	+2
P6	B	Ticket-D	4	5	+1
Median			4	5	+1

Five of six participants scored higher with AI (+1 to +2), and one (P1) scored lower (−1). The median score changed from 4 to 5, and no participant scored below 3 in either condition.

Table 5.12 breaks down accuracy by scenario to separate the effect of AI from the effect of scenario difficulty.

Table 5.12: Information search accuracy by scenario and condition, showing how the same scenario was scored under opposite conditions by different groups.

Scenario	Condition	Participants	Scores
Ticket-C	Without AI	P4, P5, P6 (Group B)	3, 4, 4
Ticket-C	With AI	P1, P2, P3 (Group A)	5, 5, 6
Ticket-D	Without AI	P1, P2, P3 (Group A)	6, 4, 4
Ticket-D	With AI	P4, P5, P6 (Group B)	5, 6, 5

Ticket-C was harder than Ticket-D without AI, with scores of 3 to 4 on Ticket-C compared to 4 to 6 on Ticket-D. AI assistance raised Ticket-C scores to 5 to 6, closing the gap with Ticket-D. On Ticket-D the AI and without-AI scores were comparable (5 to 6 with AI, 4 to 6 without).

5.2.2.2 Time

Table 5.13: Information search session time per participant and condition. All AI sessions include the multi-agent investigation time, which adds approximately 10-12 minutes before the participant begins answering.

ID	Group	Without AI	With AI	Delta
P1	A	28:17	36:39	+8:22
P2	A	27:12	35:14	+8:02
P3	A	20:44	32:34	+11:50
P4	B	09:48	29:55	+20:07
P5	B	38:59	39:31	+0:32
P6	B	30:42	33:43	+3:01
Median		27:44	34:28	+6:43

Every participant's AI session was longer than their without-AI session. The AI condition includes the multi-agent investigation described in Section 5.1.2, which executes before the participant begins answering questions and typically takes 10 to 12 minutes. The without-AI median of 27:44 is therefore the more direct measure of investigation time. The AI median of 34:28 reflects the investigation plus the automatic investigation.

P4 completed the without-AI round in 9:48, substantially faster than any other session and scoring 3 out of 6, the lowest score in the experiment. The participant reported in post-session feedback that they had rushed the investigation without consulting secondary sources. The 9:48 without-AI time is the dominant factor behind the without-AI standard deviation across participants being roughly three times the AI standard deviation.

5.2.2.3 Per-Question Analysis

Table 5.14: Information search correct answers per question across all participants (n=6 per condition). Each row pairs one question from Ticket-C with the corresponding question from Ticket-D at the same diagnostic stage. Rows marked with * pair questions that target related but not strictly equivalent stages. This is because each scenario is a real historical ticket and not every fault followed the same troubleshooting path. The totals in these rows are therefore indicative rather than a like-for-like comparison.

Q	Ticket-C	Ticket-D	Without AI (of 6)	With AI (of 6)	Δ
Q1	Problem identification	Problem identification	5	6	+1
Q2	Root cause	Root cause	5	4	−1
Q3*	Validation	Workaround	2	4	+2
Q4	Evidence cross-referencing	Evidence cross-referencing	5	6	+1
Q5	Assessment	Assessment	3	6	+3
Q6*	Resolution	Defect tracking	5	6	+1
Total			25/36	32/36	+7

AI assistance had the largest effect on the Assessment question (Q5, all six AI-assisted participants answered correctly compared to three of six without AI) and on the Validation / Workaround question (Q3, 4 vs 2). These questions required cross-referencing evidence and identifying a validated workaround, tasks where the AI's ability to correlate across DTS, TRTS, PDS, and EKSS in one pass provided an advantage over manual navigation between systems. The Root cause question (Q2) was the only one where the without-AI condition scored higher than the AI condition (5 vs 4), with two participants selecting an incorrect mechanism despite the AI having presented the correct one.

5.2.2.4 Cognitive Load

Table 5.15: Information search NASA-TLX ratings (1–21 scale). For all scales, lower values indicate less workload. The Performance scale measures self-rated dissatisfaction with own performance: a lower score means the participant felt they performed better.

		Mental	Time Pr.	Perf.	Effort	Frust.
P1 Without AI	(Ticket-D)	7	4	10	10	3
P1 With AI	(Ticket-C)	7	5	10	10	3
P1 Delta		0	+1	0	0	0
P2 Without AI	(Ticket-D)	10	10	14	14	14
P2 With AI	(Ticket-C)	10	10	14	13	15
P2 Delta		0	0	0	−1	+1
P3 Without AI	(Ticket-D)	17	17	16	21	20
P3 With AI	(Ticket-C)	3	4	2	4	5
P3 Delta		−14	−13	−14	−17	−15
P4 Without AI	(Ticket-C)	14	7	14	7	6
P4 With AI	(Ticket-D)	8	7	6	7	4
P4 Delta		−6	0	−8	0	−2
P5 Without AI	(Ticket-C)	19	14	8	18	18
P5 With AI	(Ticket-D)	13	12	10	10	10
P5 Delta		−6	−2	+2	−8	−8
P6 Without AI	(Ticket-C)	14	15	14	8	16
P6 With AI	(Ticket-D)	13	13	7	8	10
P6 Delta		−1	−2	−7	0	−6

Four of six participants reported lower workload with AI across most scales. One (P1) reported no meaningful change on any scale, and one (P2) reported essentially unchanged workload with a small increase in frustration. P3 reported the largest drop across every scale, with mental demand falling from 17 to 3, effort from 21 to 4, and frustration from 20 to 5, the largest individual reduction observed in either case.

On the Performance scale, five participants rated themselves the same or better in the AI condition. P5 rated their performance marginally worse despite scoring two points higher. Frustration decreased for four participants (P3, P4, P5, P6) and increased for one (P2). P1 reported identical frustration of 3 in both conditions, the lowest pair of scores in the experiment.

Across the six participants, the NASA-TLX sum was lower in the AI condition for four participants (P3, P4, P5, P6); P2 showed a net change of zero, and P1 a negligible increase of 1.

5.2.2.5 Qualitative Observations

Case 2's multi-agent architecture performs a fixed three-round pre-investigation (Discovery, Targeted Leads, Challenge) before returning control to the engineer. Each AI session, therefore, contained both an autonomously generated investigation report (ranked hypotheses with confidence labels and evidence citations across CSTS, TRTS, DTS, PDS, and EKSS) and the engineer's subsequent chat turns. Two consequences follow for interpreting the chat data. Prompt counts understate engagement, because a single case-ID prompt triggered roughly 8 to 12 sub-agent invocations and produced a synthesis of around one thousand words. The failure surface shifts from information-gathering to reading and hypothesis-weighting, because the multi-agent investigation has already collected the evidence.
Table 5.16 summarizes how each participant used the AI tool during their AI-assisted round. Prompt counts were extracted from the Kiro conversation log by counting user turns that carried a freshly typed prompt, excluding the multi-round tool-use round-trips that the agent performs internally.

Table 5.16: Information search AI usage patterns during AI-assisted rounds. Prompts refer to user prompts sent to the agent after the multi-agent investigation is completed. Participants are ordered by prompt count.

ID	Prompts	Usage style	Score
P2	1	One-shot oracle: submitted the case identifier only, accepted the autonomous investigation synthesis without follow-up	5/6
P4	2	Minimal dispatch: brief symptom tag followed by the case identifier; no further prompts	5/6
P6	3	High-delegation: case identifier plus two benchmark-mirrored follow-ups on root cause and recommendation	5/6
P5	4	Disciplined minimalist: brief opener, case identifier, two targeted follow-ups on fix mechanism and cross-customer spread	6/6
P1	6	Goal-directed: full symptom summary followed by narrow follow-ups on related trouble reports and version links	5/6
P3	9	Iterative stage-by-stage: one prompt per diagnostic stage, including one falsification hypothesis the agent had not produced	6/6

Prompt count did not predict accuracy within this sample. The two perfect scores came from P5 (four prompts) and P3 (nine prompts). The other four participants each scored 5 out of 6, with prompt counts ranging from one to six. Five of six

participants opened the session by submitting either the bare case identifier or a short symptom tag, delegating the initial investigation entirely to the agent. Only P3 offered a domain hypothesis that the agent had not produced (scaling down the monitoring-agent pod on the standby node only) as a falsification step. The other five let the agent's ranked synthesis stand.
Table 5.17 classifies the four wrong answers across the AI condition.

Table 5.17: Classification of the four wrong answers in AI-assisted rounds. Each row is one wrong answer.

Participant	Question	Error category
P1	Ticket-C Q3 (Validation)	Engineer anchored on own hypothesis despite the agent having presented the correct validation step
P2	Ticket-C Q3 (Validation)	Engineer transferred a hypothesis-shaped agent output to an MCQ option without requesting disambiguation
P4	Ticket-D Q2 (Root cause)	Agent ranked primary hypothesis with high confidence; engineer followed the ranking to the keyed secondary cause's detriment
P6	Ticket-D Q2 (Root cause)	Same pattern as P4: miscalibrated agent confidence in H1 despite its own caveats about missing evidence

The four wrong answers focus on one question per scenario: Q3 in Ticket-C (validation) and Q2 in Ticket-D (root cause). In each case, the wrong question was the one that probed a step or hypothesis adjacent to, but not identical with, the agent's headline claim. The two failure modes split cleanly by scenario. Ticket-C errors were engineer-side (answer transfer and anchoring on the engineer's own framing despite correct AI output). Ticket-D errors were agent-side (miscalibrated ranking, where the keyed answer corresponded to the agent's H2 rather than H1). Because the autonomous investigation delivers ranked hypotheses with explicit confidence labels, the characteristic error mode in Case 2 is the weighting of present evidence rather than missing evidence.
Several interpretive limits follow from the chat data alone. The benchmark platform did not instrument reading or scrolling behavior, so the time spent between prompts cannot distinguish deep reading of the autonomous investigation synthesis from scrolling or skimming. Per-question timestamps cover answer time as a single window that combines re-reading the synthesis, comparing MCQ options, and any out-of-chat verification. Finally, the chat log cannot confirm whether any engineer opened the CSTS, TRTS, DTS, or EKSS links that the agent cited. Absence of prompts on a topic is suggestive but not conclusive evidence that the engineer did not read the cited material externally.

Personal observations in the interviews. During the interviews, participants were asked to reflect on the two rounds. The interviews were conducted in Swedish, and the participant remarks quoted below are the authors' translations. Five of six participants said, while reflecting on their manual round, that they would have needed more information to answer confidently, naming missing logs, missing customer context, the absence of a prior ticket history, or the inability to ask a more senior colleague or the customer directly. One expressed this as "you really should be looking at log files here, but" not being able to find any to look at (P2), another as "I have nothing to hang the guess on. . . I'm missing information" (P5), and a third as "there isn't even information to search on here" (P6). By contrast, none of the six raised the same complaint when reflecting on their AI round. The agent did not provide any information that the manual condition lacked, as both rounds used the same masked ticket data. What it supplied instead was a structured synthesis of the available evidence together with ranked hypotheses and explicit citations. Whether the apparent sufficiency of information under the AI condition reflects a genuine reduction in the need for external context or a reduced awareness of what is missing is returned to in Chapter 6.

Table 5.18: Number of participants who reported, during the post-experiment interview, that they would have needed more information to answer confidently in each round. The P6 AI-round response was classified as ambiguous, as the participant asked about attaching their own log files rather than stating that the available information was insufficient.

	Manual round	AI round
Asked for more information	5	0
Did not ask	1	5
Ambiguous	0	1

5.3 RQ2: Agent Accuracy

5.3.1 Case 1: Log Analysis Agent

5.3.1.1 Multiple-Choice Accuracy

The log analysis agent was evaluated in batch mode on Ticket-A and Ticket-B (see Section 4.3.3.6). Each scenario was run six times.

Table 5.19: Log analysis agent accuracy in batch mode (6 runs per scenario)

Scenario	Scores	Mean	Avg time (s)
Ticket-A	6,6,6,6,6,6	6.0/6	69
Ticket-B	6,6,6,6,6,6	6.0/6	76

The agent achieved 100% accuracy across all 12 runs with perfect consistency, completing each scenario in approximately one minute.

In separate unguided analysis runs on Ticket-A, the agent was given no questions and produced a free-form analysis. Its output varied between sessions. In one run, it incorrectly concluded that all transactions in the transaction log had failed, missing more than 7,000 successful transactions. In the next run on the same files, it correctly identified both successes and failures. Structured prompting produced perfect consistency. Unguided prompting did not. This is the primary RQ2 observation on Case 1. The agent's reliability depends heavily on how the task is framed.

5.3.1.2 Open-Ended Analysis

Using the method described in Section 4.3.3.6, 12 trouble reports across 12 fault types were evaluated. For each, the agent received only the customer report and log files and produced a complete diagnostic analysis, scored against a predefined checklist of findings derived from the verified resolution. Scoring is strict binary per finding as Found, Missed, or Incorrect. (TR identifiers are internal Ericsson records, retained for traceability rather than external reference.)

Table 5.20: Open-ended agent accuracy across 12 trouble reports spanning 12 distinct fault types. Each TR was scored against a checklist of log-derivable findings derived from the verified TRTS resolution. Scoring is binary per finding.

#	Fault category	Findings	Score	Time (s)
1	Pod lifecycle / restart	6	6/6 (100%)	73
2	Migration timeout	5	5/5 (100%)	75
3	Thread safety	5	5/5 (100%)	54
4	Error mapping	10	6/10 (60%)	63
5	Message-bus rebalance	8	7/8 (88%)	39
6	Rollback logic	8	8/8 (100%)	48
7	Directory-service saturation	8	6/8 (75%)	63
8	Status reporting	11	10/11 (91%)	58
9	Certificate configuration	9	7/9 (78%)	52
10	Secrets-store migration	10	7/10 (70%)	62
11	Database GC pressure	12	10/12 (83%)	82
12	Library compatibility	15	10/15 (67%)	60
	Total (12 TRs)	107	87/107 (81%)	avg 61

Four of 12 trouble reports scored 100% (TRs 1, 2, 3, and 6 in Table 5.20). Of 107 log-derivable checklist findings, 87 were identified correctly, 19 were missed entirely (silent omissions), and 1 contained an incorrect claim where the agent named an adjacent component rather than the actual faulty one. The 19 missed findings cluster into six recurring failure modes analyzed in Section 6.2.1.1. Two trouble reports (TRs 1 and 2) warrant a separate note and are scored on the checklist items that remained verifiable from the available logs. The checklist could not score the root-cause item on those cases because the root-cause evidence was later found to be missing from the attached logs. Rather than reporting insufficient evidence on those root-cause items, the agent produced a confident, incorrect inference in each case: in TR 1 it guessed

that the source directory was empty when the actual root cause was a permission-denied state after a VM power-cycle, and in TR 2 it interpreted a 409 retry response as a conflict rather than a timeout-triggered race. These two off-checklist inferences, together with the one checklist-level incorrect claim, are the agent's three confident errors across the 12 trouble reports.
The agent's output included cross-file correlation, causal chain tracing, temporal reasoning, and noise filtering. Its diagnostic summaries were consistently usable as a first-pass triage input.

5.3.2 Case 2: Information Search Agent

5.3.2.1 Multiple-Choice Accuracy

The information search agent was given the same six multiple-choice questions used in the engineer experiment on Ticket-C and Ticket-D. Both the solo and multi-agent configurations were evaluated in batch mode, where all six questions are sent in a single prompt, directly parallel to the log analysis evaluation in Table 5.19. Each scenario was run three times per configuration, for a total of 12 runs.

Table 5.21: Information search agent accuracy in batch mode (3 runs per scenario per configuration).

Configuration	Scenario	Scores	Mean	Avg time (s)
Solo (batch)	Ticket-C	6,6,6	6.0/6	165
Solo (batch)	Ticket-D	6,6,6	6.0/6	207
Multi (batch)	Ticket-C	6,6,6	6.0/6	623
Multi (batch)	Ticket-D	6,6,6	6.0/6	622

Both configurations achieved 100% accuracy across all 12 runs with perfect consistency. The solo agent completed each scenario in roughly three minutes, while the multi-agent controller took roughly ten minutes. On this closed-form multiple-choice task, where the answer options constrain the search space, the multi-agent's three-round investigation produces the same accuracy as the solo agent at approximately four times the wall-clock cost. The multi-agent's advantage is investigative breadth on open-ended cases where the correct answer is not one of four given options. That advantage is assessed separately in the 30-case corpus evaluation that follows.

5.3.2.2 30-Case Corpus Evaluation

The information search agent was evaluated on the 30-case corpus described in Section 4.3.3.6, stratified into 10 low-priority, 10 medium-priority, and 10 high-priority cases. Each case was executed three times against the multi-agent `controller` configuration, giving 90 total runs. The solo agent was not evaluated on this corpus. The scoring harness measures compliance with the multi-agent's three-round dispatch contract, which the solo agent does not implement. Designing a separate evaluation harness for the solo agent was not feasible within the project timeline. This limits the architecture comparison to the batch-mode multiple-choice results in Table 5.21, and

a full corpus-level comparison remains future work. The three repeats allow separation of agent behavior from run-to-run LLM sampling variance. Each run was scored by the `evaluate_v5.py` harness on the five deterministic metrics and three structural checks defined in Table 4.5. All 90 runs scored 1.0 on the Convergence check, meaning every dispatch round introduced at least one previously unseen intent.
One run of 90 (run 330, case Customer-Ticket-M7) dispatched a fourth round rather than the three prescribed by the architecture. The remaining 89 runs complied with the three-round contract. One run of 90 (run 268, case Customer-Ticket-M9) contained a weak-evidence round, and the agent adapted on the following round, giving Evidence Recovery Fidelity 1.0 on that run. The other 89 runs had no weak-evidence rounds and ERF was reported as not applicable.
Table 5.22 reports the overall performance across all 90 runs. Dispatch Efficiency and Redundancy are complementary views of the same ratio and are reported together. The agent achieved high scores on operational metrics (Actionability 0.92, Dispatch Efficiency / Redundancy 0.90) and a moderate score on diagnostic convergence (0.73), while source recovery was lower and more variable (0.47, standard deviation 0.25). A Source Recovery of 0.47 means that, on average, the agent recovered only half of the evidence references present in the reference run. The standard deviation of 0.25 indicates substantial case-to-case variation.

Table 5.22: Information search agent accuracy across 90 runs (30 cases × 3 repeats). Each metric is reported individually and no composite score is computed. Meimandi et al. [33] show that privileging one evaluation dimension while omitting others produces measurement imbalance, and individual reporting avoids this risk. Dispatch Efficiency and Redundancy coincide numerically by construction.

Metric	Mean	Std. dev.	Min	Max
Source Recovery	0.47	0.25	0.08	1.00
Diagnostic Convergence	0.73	0.27	0.00	1.00
Actionability	0.92	0.07	0.75	1.00
Dispatch Efficiency	0.90	0.06	0.67	1.00
Redundancy	0.90	0.06	0.67	1.00
Convergence (check)	Mean 1.00; 90/90 runs scored 1.00			
Evidence Recovery Fidelity (check)	1/1 = 1.00 on the single applicable run; 89/90 n/a			
Round count (check)	89/90 had exactly 3 dispatch-bearing rounds; 1/90 had 4			

Table 5.23 breaks down the same metrics by difficulty tier. Operational metrics were stable across all three tiers, varying by less than 0.02. Diagnostic convergence was highest on the high-difficulty tier (0.82), consistent with high-difficulty cases having more documented root-cause evidence for the agent to align with. Source recovery was lowest on medium-difficulty cases (0.39) and comparable on the low and high tiers (0.51 and 0.51), indicating that recall is not monotonic in tier difficulty.

Table 5.23: Information search agent accuracy by difficulty tier (N=30 runs per tier, each case run three times).

Difficulty	SrcRec	DiagConv	Action	D.Eff	Redun
Low	0.51	0.67	0.92	0.90	0.90
Medium	0.39	0.69	0.92	0.89	0.89
High	0.51	0.82	0.93	0.90	0.90

Table C.1 (Appendix C) reports per-case stability as the mean and standard deviation of source recovery and diagnostic convergence between the three repeats of each case. 17 out of 30 cases showed a source-recovery standard deviation at or below 0.10, indicating reproducible recall. The remaining 13 cases showed larger variation, with the worst (Customer-Ticket-L2) producing three source-recovery scores of 1.00, 1.00, and 0.08 across repeats, a standard deviation of 0.53 on a single case. Diagnostic-convergence variation was concentrated on five cases (Customer-Ticket-L2, Customer-Ticket-L6, Customer-Ticket-L7, Customer-Ticket-M5, Customer-Ticket-M10), each with $\sigma \geq 0.38$.

The information search agent has access to the same data sources (CSTS, TRTS, DTS, PDS, EKSS, and RHS) that contain the verified case resolutions. The accuracy results, therefore, measure the agent's ability to retrieve and correctly align evidence to the known resolution, not its ability to independently reason to an answer. This is a different measurement from the log analysis accuracy test, where the agent had to analyze raw log files and infer findings. For the same reason, the 0.47 Source Recovery and 0.73 Diagnostic Convergence reported here are not directly comparable to the 81% finding rate reported for Case 1 in Section 5.3.1. Case 1 measures whether the agent identifies expert-authored findings against a known ground truth, whereas Case 2 measures agreement between two free-form investigation reports.

Three properties of the scoring pipeline temper the interpretation of these numbers. Source Recovery measures recall against the raw-run reference output. It is not a classification accuracy. When the raw report produces no parsable evidence references, Source Recovery and Diagnostic Convergence return 1.0 by convention rather than being undefined, and such vacuous scores are included in the reported means. The Evidence Recovery Fidelity check fired on only one of 90 runs, limiting what can be concluded about the agent's behavior on weak-evidence rounds.

With those caveats stated, the results indicate that the agent reliably dispatches a well-structured investigation (operational metrics ≥ 0.89, Convergence 1.00 on every run, three-round contract satisfied on 89 of 90 runs) but recovers only roughly half of the evidence references the raw-run reference produced, with substantial per-case variation.

Table 5.24 reports how the agent directed its investigation across the six underlying data sources, aggregated over 90 runs. The 1084 unique dispatches split across three sub-agent roles (`knowledge` 47%, `CSTS` 29%, `devtools` 24%) and reference six data sources with substantially different prevalence. Five of the six platforms (DTS, CSTS, TRTS, PDS, EKSS) are referenced on every single one of the 90 runs. RHS is the sole conditional platform, referenced on 29 of 90 runs and never consulted on 14 of 30 cases. Usage is tier-invariant: the mean dispatches per platform per run

vary by at most ±0.06 across the low-, medium-, and high difficulty tiers, meaning the agent devotes the same investigative breadth to an easy case as to a hard one. Cross-run reproducibility, in contrast, differs by a factor of three between platforms. TRTS, where queries are case-anchored on a specific trouble-report identifier, is the most reproducible (mean $\sigma = 0.26$ across three repeats, consistent use in 17 of 30 cases). CSTS, where queries are broad cross-customer searches for symptomatically related cases, is the most variable (mean $\sigma = 0.69$, consistent use in only 6 of 30 cases). The procedural decision about *which* platforms to consult is almost fully stable across repeats; the decision about *how many times and with what specificity* to consult each one is not, and the variance is concentrated on the platform with the broadest query scope.

Table 5.24: Platform-usage and reproducibility across 90 runs (30 cases × 3 repeats). "Mean dispatches/run" is the average number of dispatches mentioning that platform per run. "Mean σ" is the average across the 30 cases of the within-case standard deviation over 3 repeats. "Consistent" counts cases whose three repeats produced identical non-zero usage; "Never used" counts cases whose three repeats all produced zero dispatches for that platform.

Platform	Mean dispatches/run	Mean σ	Consistent	Never used
TRTS	2.99	0.26	17/30	0/30
RHS	0.40	0.35	2/30	14/30
PDS	2.78	0.46	8/30	0/30
DTS	4.06	0.51	10/30	0/30
EKSS	1.79	0.58	7/30	0/30
CSTS	3.20	0.69	6/30	0/30

Chapter 6

Discussion

This chapter interprets the results in relation to the research questions and connects the findings to the related work reviewed in Chapter 3.

6.1 RQ1: Impact on Performance and Cognitive Load

6.1.1 Log Analysis

Ticket-B was consistently harder than Ticket-A across both conditions. The per-question analysis (Table 5.7) provides the most direct comparison because it pools all six participants per condition regardless of scenario. Q2 (timeline) and Q3 (recovery status) showed the largest difference, with all six AI-assisted participants answering correctly compared to three of six without AI. These questions required locating specific events within large log files, where the AI's targeted search reduced the retrieval cost described by Pirolli and Card's information foraging framework [8]. Q4 (data loss or root cause detail) was the only question where the without-AI condition outperformed the AI condition (2 vs 1 correct), and Q6 (final assessment) showed no difference.

The Q4 result is examined in detail below. All three Group B participants who attempted Ticket-B Q4 with AI made the identical error. They selected "incoming provisioning requests and alarm raise notifications" when the AI had described the missing data as "fault indications could not be published" and "processing logs could not be stored." The correct answer used the terms "audit logs" and "alarm clear notifications," requiring the engineer to map between the AI's technical terminology and the answer option's functional terminology. The two options shared the "alarm" wording but reversed the polarity ("clear" vs "raise") and swapped the first clause, so the mapping crossed both the data category and the alarm direction. This systematic pattern suggests that when the AI's output and the answer options use different vocabulary for the same concept, engineers may fail at the mapping step rather than at retrieval.

Median session time was 4 minutes 17 seconds longer in the AI-assisted condition (28:05 vs 23:48). This difference should be interpreted cautiously, given the small sample, the scenario confound, and technical disruptions affecting four of six sessions (two of which required time corrections). The AI interaction introduces steps that are not part of manual analysis, such as formulating prompts, waiting for responses, and reading the output. However, the two participants who used the AI most interactively

(P2, P3) achieved the highest accuracy, suggesting a time-accuracy trade-off rather than pure overhead. The time data does not support the claim that AI assistance reduces diagnostic time in this experimental context.

The cognitive load data revealed what we term the *false ease* pattern, in which perceived workload drops substantially while accuracy does not increase proportionally. P5 and P6 both exhibit this pattern (P5: -2, P6: $+1$), reporting substantially lower mental demand with AI without a matching gain in accuracy. Ahuja et al. [17] reported low cognitive workload (NASA-TLX 2.53/7.0) alongside improved performance in their VIGIL deployment, demonstrating that reduced cognitive load can co-occur with better outcomes. The contrast with our data, where reduced cognitive load co-occurred with unchanged or even lower accuracy, suggests that the false-ease risk depends on the interaction design, not just the tool's capabilities.

The AI usage data (Table 5.9) reveals that prompt quantity alone did not predict performance. P3 and P5 both sent eight prompts but scored 5/6 and 2/6, respectively. P3 conducted a systematic walk-through of the diagnostic stages, maintaining an independent diagnostic plan and using the AI to execute specific steps. P5 asked broad questions and accepted answers without follow-up. This maps onto Feng et al.'s [4] distinction between detailed delegation, where the engineer maintains control through specific instructions, and over-reliance, where the engineer defers to AI output without critical evaluation. The false ease pattern observed for P5 may be the quantitative outcome of over-reliant delegation. Perceived effort drops because the engineer has stopped performing the critical evaluation that makes AI output useful. The variation in usage styles is consistent with the Naturalistic Decision Making framework [9]: engineers with stronger diagnostic mental models used the AI within an existing strategy, while those without such models deferred to its output without independent evaluation.

The wrong-answer analysis (Table 5.10) revealed that two of the 13 errors involved AI failures: one was a factually incorrect analysis, and the other involved the AI giving contradictory responses across turns. The remaining eleven were human errors, namely misreading answer options, poor prompting, failing to consult the AI, or drawing incorrect conclusions from correct output. This error distribution is consistent with the shift from producing output to evaluating it. The human analog of the MCP-Atlas [16] finding that models most commonly fail by not calling tools is that engineers do not ask the AI about the specific issue they are struggling with (two of 13 errors).

6.1.2 Information Search

The AI-assisted condition produced higher accuracy on the information search task for five of six participants, with a median of 5 out of 6, compared to 4 out of 6 in the manual condition. The one participant whose score dropped (-1) scored 6 out of 6 without AI, indicating a ceiling-adjacent score on both sides rather than AI-induced error. No participant scored below 3 out of 6 in either condition. The effect on accuracy is therefore directional and consistent but small in absolute terms. As in the log analysis case, the sample is too small to support inferential claims, and the AI-assistance effect is reported as an observed pattern within this cohort rather than as a general finding.

The accuracy gap was unevenly distributed across the two scenarios. Ticket-C was substantially harder without AI (3–4 out of 6) than with AI (5–6 out of 6), while Ticket-D produced comparable scores in both conditions (4–6 without, 5–6 with). The AI assistance closed the scenario-hardness gap rather than uniformly raising all scores. This is consistent with the information foraging account of Pirolli and Card [8]. The benefit of a structuring tool is largest where the engineer's own foraging cost is highest, and negligible where the original source is already well-organized.

Prompt count did not predict accuracy within this sample. The two perfect scores came from the participants who sent nine (P3) and four (P5) prompts, while participants who sent one, two, three, and six prompts each scored 5 out of 6. This pattern is not comparable to the log analysis case, where prompt count varied with engagement style on an interactive chat interface. The Case 2 agent performs a three-round autonomous investigation before returning control to the engineer, so a single case-identifier prompt still triggers the full synthesis. The prompt count measures how much the engineer interrogated the synthesis, not how much the agent worked. This decouples the prompt-count-as-effort heuristic that was applicable in the log analysis case.

The four wrong answers in the AI condition concentrated on one question per scenario, with distinct failure mechanisms in each. On Ticket-C, the wrong question was Q3 (the validation step), and both errors were on the engineer's side. The engineer committed to a personal hypothesis despite the agent having presented the correct step. On Ticket-D, the wrong question was Q2 (the root cause), and both errors were agent-side. The agent ranked a primary hypothesis with high confidence, and both engineers followed the ranking without requesting discriminating evidence, even when the agent's own text flagged missing evidence. The two patterns are distinct failure modes under different conditions, but they share a root cause. The engineer did not use the chat interface to challenge the synthesis after reading it. This extends the detailed-delegation distinction of Feng et al. [4] to a multi-agent investigation setting, where the agent supplies a finished narrative, and the cue to delegate with follow-up questions is reduced precisely because the synthesis reads as complete.

Search competence also varied substantially across the six participants. In manual rounds, strategies ranged from a systematic, stage-by-stage query of available sources (P3) to a single glance at the ticket record with no secondary search (P4, P6). Four of six participants named in-house search across CSTS, TRTS, and related systems as a pain point, one describing the normal workflow as "almost impossible" (P2) and another dismissing the vendor-provided knowledge base outright (P3). The AI condition's observed benefit is therefore partly a benefit relative to a search task that engineers themselves find structurally difficult, rather than over a competent baseline. This does not invalidate the accuracy pattern but constrains its interpretation.

A notable pattern in the interviews was the asymmetry in perceived information sufficiency between the two conditions, recorded in Table 5.18. Five of six participants said, during their manual round, that they would have needed more information to answer confidently. In the AI round, none of the six raised the same complaint. Both rounds operated on the same masked ticket data, so the agent did not provide any information that the manual condition lacked. What it provided was a structured synthesis of the same evidence, with ranked hypotheses and explicit citations.

Engineers stopped saying they needed more information once the AI summarized what was already in front of them. The data was the same. Only the presentation changed. This matches what Parasuraman and Riley [39] describe as automation bias, in which users rely on an automated tool rather than checking the underlying evidence themselves. The Case 2 agent is consistent with this effect at the presentation layer. Engineers stopped noticing what was missing, not because the missing information had been supplied, but because the structure of the output closed the visible gap.

Two further nuances should be recorded. The time comparison, with the AI condition 6 minutes 43 seconds longer on average, is not directly comparable to the manual condition because the AI session includes the 10 to 12 minutes of autonomous investigation that executes before the engineer begins answering. The pure engineering investigation time is similar across conditions, and the headline time difference largely reflects the fixed cost of the autonomous investigation. The cognitive load data show reductions across all five NASA-TLX scales for the AI condition at the median, with one participant (P3) reporting the largest observed drop in the study (mental demand 17 to 3, effort 21 to 4, frustration 20 to 5). These reductions correspond to the qualitative picture of a synthesis that engineers find easier to consume than a raw investigation, but, as in the log analysis case, the relationship between reduced perceived load and actual performance is not consistent. P1 reported an unchanged workload and scored one point lower with AI, showing no clear relationship between the two.

6.2 RQ2: Agent Accuracy

6.2.1 Log Analysis

The structured multiple-choice test gave 100% accuracy. The open-ended evaluation gave 81%. This 19-percentage-point gap, together with the six failure modes identified below, is the principal RQ2 finding on Case 1. The agent is a strong evidence retriever and a weak evaluative reasoner. The structured test supports the conclusion that the diagnostic information is present in the logs and can be extracted. The open-ended evaluation across 12 trouble reports (81% of log-derivable findings identified) shows that the agent produced outputs consistent with causal chain tracing, noise filtering, and actionable recommendations. The missed findings do not fall into a single category but cluster into six recurring failure modes analyzed in Section 6.2.1.1.

The gap between agent-alone accuracy (100% structured, 81% open-ended) and engineer-with-AI accuracy (median 4/6) is an observation relevant to RQ2, though the comparison is not direct. The agent received all six questions simultaneously, while the engineers answered sequentially without seeing future questions. With this caveat, the gap suggests that engineers do not fully use the agent's capability. They ask partial questions, work manually alongside the AI, and sometimes do not consult the AI about the specific issue they are struggling with.

The inconsistency observed in unguided analysis runs, where the agent produced contradictory conclusions on the same data across sessions, qualifies the structured-mode

result. On one such run, the agent declared that all transactions in the transaction log failed, even though more than 7,000 were successful. The next run on the same inputs correctly identified both successes and failures. Structured prompting produced perfect consistency, while open-ended prompting did not. This suggests that the reliability of AI-assisted diagnostics depends as much on how the engineer frames the task as on the model's underlying capability.

6.2.1.1 Failure-mode taxonomy

The 19 missed findings and the 1 contradicting claim cluster into six recurring failure modes across 12 trouble reports. These modes correspond to specific categories of diagnostic reasoning that the agent systematically handles less reliably than others.

Classification and labeling gap. Evaluating whether a label fits an event requires semantic judgment beyond what the log text itself provides. The agent describes what the log says but does not evaluate whether what the log says is appropriate. When a fault code, severity level, or error label misdescribes the event it reports, the agent typically fails to surface the mismatch. This pattern appears in TR 4 (a replay-mode fault code returned on a GET request), TR 5 (ERROR-level logging on a routine message-bus rebalance event that should be INFO), TR 7 (node communication failures mapped to a generic System error instead of the specific Communication error), and TR 9 (cert mismatch correctly described but not explicitly classified as a configuration rather than code fault).

Differential-evidence gap. The agent reads entries sequentially and summarizes what is present. It does not form hypotheses and then search for corroborating or contradicting evidence across entries. In TR 4 the agent listed all four affected clients but did not notice that one client produced a timeout exception rather than a certificate-authorization error, which itself proves that the client's identifier was in the certificate's authorized list. In the same TR the agent did not surface that a key context field was empty on every one of 82 log entries, directly falsifying the stated precondition error message. Differential evidence, where one entry tells a distinguishing story, is a primary diagnostic tool for human engineers and a consistent weakness for the agent.

File-scoping gap. The agent prioritizes files whose names match the customer-reported symptom and underexplores supplementary files.
In TR 4, the agent processed two of three JSON artifacts but skipped the one containing the lifecycle context for the failed operation. In TR 8, the agent analyzed the main process log but did not open the secondary log directory that contained the JSON parse error, which was the direct evidence that the service swallowed the failure. In real support tickets, the root cause is often not in the file the customer points to.

Off-checklist wrong inference. When root-cause evidence is not in the attached logs, the agent does not state "evidence insufficient." It produces a plausible alternative diagnosis. In TR 1 the actual root cause (permission-denied on overwrite after

VM power-cycle) is not visible in the supplied logs. The agent guessed that the source directory was empty. In TR 2 the 409 retry response is caused by a timeout-triggered race condition. The agent interpreted it as a genuine state conflict. Both guesses were wrong and both recommendations would have steered support away from the actual defect. The agent's output reads the same whether confidence is well-supported or fabricated.

Numerical and arithmetic errors. Specific counts can be wrong even when the surrounding narrative is correct. The checklist scores diagnostic findings only, not numerical precision, so this category captures arithmetic errors that the checklist cannot flag. TR 6 scored 8/8 on the checklist but stated "zero succeed" when three SEARCH operations had returned successful, and "0 out of 31 southbound operations succeeded" when the actual total was 3 of 25. The agent's impressionistic counting is consistent with its text-level reading but does not match the underlying data when engineers cross-check.

Echo-based remediation. The agent reproduces remediation hints from error messages without evaluating whether they apply to the specific defect. In TR 5 the agent suggested tuning a consumer-timeout parameter (the generic remedy suggested by the message bus's own error message) when the actual fix is to change the log level. Following this echo-based advice would optimize the wrong configuration. The defect would remain.

Across these six failure modes, a common pattern is observable. The agent is strong at *describing* logs and weak at *evaluating* them. For retrieval-style analysis, the agent substantially reduces the manual effort required. For diagnoses that require judgment about whether labels, classifications, or recommendations fit the underlying events, human review remains essential. This aligns with Feng et al.'s [4] observation that the engineer's role shifts from producing output to evaluating AI-generated output, which is precisely what the agent itself does not perform.

6.2.2 Information Search

The Case 2 agent-accuracy evaluation produces a different answer to RQ2 than the Case-1 evaluation, and the difference is not cosmetic. The Case 1 figures (81% of log-derivable findings, with diagnostic quality metrics against a human-authored checklist) measure whether the agent identifies the correct answer on a task with a known ground truth. The Case 2 figures (Section 5.3.2) assess whether the masked run aligns with the unmasked run on evidence references, category assignments, and operational processes. The raw run is a self-generated reference, not a human-verified ground truth, so Case 2 reports self-consistency under information loss rather than classification accuracy. The structural checks (Convergence 90/90, Round count 89/90, Evidence Recovery Fidelity 1/1 on the single applicable run) confirm that the architecture contract of a three-round multi-agent investigation holds across almost every run, but they do not, by themselves, carry performance information.

The principal result in Case 2 RQ2 is a split between the reproducibility of the investigative process and that of the retrieved evidence. The operational metrics

(Actionability, Dispatch Efficiency and Redundancy) show per-case standard deviations at or below 0.14, and the structural checks pass on every well-formed run. In contrast, Source Recovery shows a standard deviation of 0.25 across 90 runs and reaches 0.53 per case, while Diagnostic Convergence shows a standard deviation of 0.27. The same case run three times produces a near-identical investigation shape but different evidence sets and, occasionally, different category assignments. This is consistent with the literature on LLM sampling variance. Procedural scaffolding is more stable than factual recall across samples, and structure-level self-consistency does not guarantee content-level self-consistency. The practical consequence for support engineering is that the agent can be trusted to follow a process on the third try the same way it followed it on the first, but cannot be trusted to surface the same evidence twice.

The platform-level analysis in Table 5.24 sharpens this claim into a mechanism. The reproducibility split is not uniform across the six data sources. It concentrates on the platforms with the broadest query scope. TRTS, where the agent queries a specific trouble-report identifier extracted from the raw case, shows the lowest run-to-run variance ($\sigma = 0.26$, identical usage on 17 of 30 cases). CSTS, where the agent issues open-ended cross-customer searches for symptomatically related cases, shows the highest variance ($\sigma = 0.69$, identical usage on only 6 of 30 cases). The variance tracks query breadth, not platform reliability or data availability. The same agent, with the same prompt, on the same case, produces different CSTS dispatch counts on different runs because the LLM's sampling drives what the broad search looks like.

This connects the Ticket-D failure mode observed in RQ1 to the variance observed in RQ2. Both wrong answers in Ticket-D came when engineers accepted the agent's top-ranked hypothesis without challenging it, and the keyed correct answer corresponded to the agent's second-ranked hypothesis. The hypothesis ranking itself is produced by the agent's CSTS-led cross-customer search, which RQ2 identifies as the highest-variance platform. A run that draws one set of CSTS evidence produces one ranking. A different draw on the same case would plausibly produce a different ranking. The two findings are therefore not independent. Engineers accepted a ranking that the agent would not necessarily reproduce on a rerun.

The three-retest evaluation design made this split observable. A single-run evaluation on the worst-case Customer-Ticket-L2 would have reported either a Source Recovery of 1.00 (perfect) or of 0.08 (nearly absent), depending on which draw it caught. Reporting the mean of 0.69 with a standard deviation of 0.53 is the honest summary, and it is a summary that a single-run evaluation cannot produce. On this basis, the three-repeat structure is defended as a minimum, not a necessity. Further increasing the repeat count would tighten the variance estimate on cases like Customer-Ticket-L2 where the three samples straddle opposite ends of the range. For downstream deployment, this implies that a single agent invocation is not a reliable indicator of what it will find in that case. A self-consistency or ensemble strategy is required to smooth the evidence recall layer before the output can be acted on.

Two further hedges on these numbers should be stated explicitly. Actionability reaches 0.92 on average across 90 runs, but the metric is implemented as a regex-based template-compliance check across eight items, such as "ordered steps" and "documentation reference," rather than as a content-quality rating. A uniform output shape

is sufficient to achieve a high score. Similarly, Diagnostic Convergence uses a flat, keyword-based category classifier rather than the hierarchical taxonomy proposed by the cited framework, and this classifier has not been validated against expert judgment. These two issues are documented in the construct validity chapter and do not change the split between stable operational metrics and variable evidence metrics, but they constrain what "0.92 Actionability" and "0.73 Diagnostic Convergence" can be read to mean. Within those constraints, the picture is internally consistent. The agent performs the procedural part of information search reliably and the evidence-retrieval part variably, which is the same profile that RQ1 showed from the engineer's side, where minimalist prompting still produced near-ceiling accuracy because the procedural layer was delivered autonomously and reliably on every run.

One architectural direction, motivated by the data above but not evaluated in this study, is a meta-investigation layer on top of the existing pipeline. The multi-agent investigation of Section 5.1.2 could be executed five or six times in parallel on the ranking. A different draw on the same case, and a second multi-agent instance could consume those reports as input. The reconciliation would live inside this final agent: it would identify points of agreement and disagreement across the N reports, re-examine cited evidence where the reports diverge, and produce a single synthesis with explicit agreement counts and validated supporting evidence. This directly addresses the Ticket-D failure mode observed in RQ1, where engineers accepted a single top-ranked hypothesis without requesting discriminating evidence. A final agent that reports "H1 appeared in 3 of 6 runs, H2 in 3 of 6" forces the discrimination step that the current single-run interface does not cue. Two caveats temper the expected benefit. The N runs share the same model, prompts, and tool access, so only random sampling variance would be reduced. Correlated failures would not. The final agent is itself an LLM subject to the same evaluative failure modes the Case 1 taxonomy in Section 6.2.1.1 describes.

6.3 Practical Implications

The variation in AI usage patterns and the dominance of human errors (11 of 13) suggest that providing the tool without guidance on effective interaction strategies is insufficient. Onboarding should demonstrate the difference between systematic, progressively specific prompting (which produced the highest scores) and passive acceptance of AI output (which produced the lowest).

The binary CSV file that silently broke grep for manual users was handled transparently by the AI, suggesting that AI-assisted tools may also reduce data format friction, in addition to their diagnostic reasoning role. This format-agnostic data access is a practical benefit worth designing for.

The false ease risk observed in two participants (P5 and P6), where AI reduced perceived difficulty while not improving or even reducing accuracy, echoes Feng et al.'s [4] finding that engineers accept AI suggestions without meaningful review when the output appears to work. Organizations should not rely on subjective satisfaction metrics alone to evaluate AI tool effectiveness. Objective accuracy measures should accompany any deployment assessment.

The systematic misreading on Ticket-B Q4, where three participants selected the

wrong option despite the AI stating the correct answer, suggests that interface design should highlight counterintuitive findings to slow down engineers' review at critical decision points.
The information search case revealed a form of automation bias. Five of six participants reported needing more information in the manual round, yet none raised the same concern in the AI round, even though both rounds used the same masked data. The structured synthesis changed what engineers felt they had, not what evidence was available. Interfaces that deploy similar synthesis features should explicitly surface what evidence is missing or unverified, not only what was found, to counteract this effect.
The Ticket-D Q2 errors, where two engineers followed the agent's top-ranked hypothesis despite its own caveats about missing evidence, suggest that ranked output encourages engineers to accept the top-ranked hypothesis without evaluating alternatives. A deployment interface could mitigate this by requiring the agent to present discriminating evidence between its top two hypotheses before the engineer commits to one.
The multi-agent investigation accommodated both systematic users (P3, who interrogated the synthesis across nine prompts) and minimalist users (P2, who accepted the synthesis without follow-up). Both achieved high scores. However, the minimalist pattern carries the same false-ease risk observed in the log analysis case if the synthesis is incorrect. Deployment should pair the autonomous investigation with a prompt that asks the engineer to identify at least one gap or disagreement before proceeding.
The agent-accuracy evaluation showed that evidence recall varied substantially across repeated runs on the same case (Source Recovery standard deviation up to 0.53). A single invocation of the agent is therefore not a reliable assessment of what it will find. A self-consistency or ensemble strategy could smooth evidence recall across multiple runs before results are presented to the engineer. Over time, persistently reduced mental demand may also reduce the learning that comes from manual investigation, creating a deskilling risk if engineers stop practicing without AI support [39].

6.4 Research Implications

The dominance of human errors (11 of 13 in Case 1, 2 of the 4 AI-condition wrongs in Case 2, with the other 2 attributed to agent ranking miscalibration) identifies the interaction between engineer and tool as a research target in its own right, distinct from agent capability. Benchmark performance does not predict field performance when the translation from agent output to engineer action introduces its own error surface.
The split between stable operational metrics and variable evidence recall in Case 2 ($\sigma = 0.25$ on Source Recovery) suggests that evaluation frameworks for agentic systems should report procedural and content reliability as independent axes rather than as a single composite.
The divergence between subjective workload and objective accuracy, observed in both cases, cautions against using NASA-TLX and similar subjective measures as standalone acceptance criteria without accompanying objective performance measures.

Chapter 7

Threats and Validity

This chapter examines the threats to validity of this study, following the classification proposed by Runeson and Höst [40]: reliability, internal validity, external validity, and construct validity.

7.1 Reliability

The experiment was administered through a custom benchmark application that automatically recorded answer selections, per-question times, and NASA-TLX ratings, reducing the risk of manual data entry errors. AI chat transcripts and terminal commands were saved for all participants, providing a traceable record from raw observations to reported findings.

Four of six sessions with the participants experienced technical interruptions. These were 8.5 minutes of tool downtime in one AI session, a backend restart between questions in another, a keyboard layout incompatibility and input lag during a remote session, and a brief application crash before resuming in a fourth. In all cases except the remote session (where keyboard lag persisted), the experiment was paused during the interruption and resumed normally. All sessions were recorded on screen and no data were lost. Times were corrected where the disruption was clearly separable from the task, and all corrections are documented in the results.

The agent accuracy tests were run multiple times per scenario (6 runs for log analysis, 3 runs per scenario per configuration for information search) to assess consistency. The log analysis agent was perfectly consistent in batch mode (12 of 12 runs at 6/6). The information search agent was also perfectly consistent on the engineer-experiment MCQs in both the solo and multi-agent batch configurations (12 of 12 runs at 6/6). The 30-case corpus evaluation quantified substantial case-level variance in evidence recall. Source Recovery had $\sigma = 0.25$ in 90 runs, reaching $\sigma = 0.53$ per case. Per-case figures appear in the Appendix C.

The authors were present during every session, and participants knew they had designed the AI tool, which can bias subjective workload ratings and verbal feedback. No specific mitigation was applied.

7.2 Internal Validity

The within-subjects crossover design controls for individual differences by having each participant serve as their own control. However, the two scenarios differ in

difficulty in both cases. In Case 1, Ticket-B was consistently harder than Ticket-A across all conditions. In Case 2, Ticket-C was harder than Ticket-D without AI (scores of 3 to 4 compared to 4 to 6). The crossover design addresses this asymmetry by having both groups attempt each scenario under opposite conditions, and the balanced group assignment (three in Group A, three in Group B) ensures equal representation across conditions.

Kitchenham et al. [41] argue against crossover designs in software engineering, citing carry-over effects and the difficulty of analyzing paired data. At the scale of this study ($n \approx 6$ per case), the recommended alternative (a between-subjects design with enough participants per cell) is not practical, and Wohlin et al. [22] note that crossover is used in software engineering precisely for this small-sample trade-off. A 2024 survey by Frattini et al. [42] confirms that the design remains common in the field, while cautioning that only 29.5 % of crossover-specific validity threats are properly addressed across the surveyed studies.

Frattini et al. [42] identify three threats specific to crossover designs. Maturation is mitigated by a practice round before the measured rounds and consistent instructions across conditions. Optimal sequence is mitigated by varying the scenario between rounds and reporting results per scenario, so scenario difficulty cannot be confounded with the AI-assistance effect. Carry-over is mitigated structurally because the two rounds use different faults, so evidence from the first round cannot prime the engineer for the second.

Scenario order was fixed (Scenario 1 first, Scenario 2 second) so that every participant encountered the easier scenario in the first round. Condition alternates between groups, so at the group level each scenario is seen under both conditions, but for any individual participant, the effect of AI assistance is confounded with the effect of warming up on a second task. Separating the two would require a four-group design, which the sample of six participants per case does not support.

The Case 2 manual-round scores are consistent with this round-2 position effect, which Frattini et al. [42] identify as the "optimal sequence" threat. Ticket-C, completed first by Group B, scored 3, 4, 4; Ticket-D, completed second by Group A, scored 6, 4, 4. The participants answering Ticket-D manually had already completed Ticket-C with AI and had roughly thirty minutes of exposure to the benchmark interface and the ticket structure. The crossover design absorbs this effect for the AI-versus-manual comparison, but Case-2 scenario-difficulty statements should be read as the difficulty of each case given its position rather than as an intrinsic property.

Participants were instructed to use the AI tool as much as they felt comfortable with, while encouraged to try it slightly more than they normally would. This instruction was intentionally open-ended to capture natural usage patterns, but it meant the AI-assisted condition varied substantially across participants, from three prompts to 10 in Case 1 and from one to nine in Case 2. The observed differences in accuracy may partly reflect differences in AI usage intensity rather than AI availability alone.

The AI condition in the Case 2 experiment includes the 10 to 12 minutes of autonomous multi-agent investigation that executes before the engineer begins answering. "Session time" therefore measures different activities in the two conditions: engineer-driven investigation in the manual round, and engineer-driven investigation plus a fixed autonomous investigation cost in the AI round. The comparison of

raw session times is not apples-to-apples, and the results section reports this caveat alongside the numbers.
The multiple-choice format constrains the assessment to predefined answer options. One Case 1 question's correct answer described the message bus pods as "restarted" when the log evidence shows the pods maintained their original age. The helm upgrade disrupted the service without recreating the pods. The correct answer remains the best option among the four alternatives, but the wording may have introduced ambiguity. Four of six participants answered this question correctly.
Participants were aware that the authors had designed the AI tool, which may have biased NASA-TLX ratings toward more positive values through social desirability. The effect, if present, would inflate the perceived benefit of the AI condition.
Sessions were conducted over several weeks with no formal non-disclosure instruction. No indication of scenario contamination between participants was observed during the sessions.

7.3 External Validity

All participants were support engineers from a single team working on the same product at Ericsson. The findings may not generalize to other support teams, other products, or other organizations. However, the product serves telecommunications customers globally, and the diagnostic tasks (log analysis, information search across fragmented tools) are representative of support engineering work across the telecommunications industry.
The sample size of six participants is small, typical of industrial experiments with specialized practitioners, but insufficient for statistical generalization. The results are reported using descriptive statistics (medians and per-participant deltas) rather than inferential tests. The per-participant reporting made individual interaction patterns observable at a level of detail that group-level statistics alone would not provide, such as the systematic vocabulary mismatch on Ticket-B Q4 where all three Group B participants made the identical mapping error despite correct AI output. The Wilcoxon signed-rank test was not applied due to the small sample.
Participants volunteered after being invited through the support organization. Volunteers are more likely to be interested in AI tools than engineers who declined, potentially inflating the measured AI effect relative to a randomly selected sample. A random draw was not feasible within the timeframe of this thesis.
Three participants completed the experiment remotely via screen sharing, while three completed it locally. The remote sessions introduced additional friction (screen lag, keyboard layout issues, scrolling difficulties) that was not present in the local sessions. This difference in experimental conditions is a threat to comparability across participants.
The agentic solutions were built on a specific AI platform (Kiro by Amazon) using models available in April 2026. The rapid evolution of LLM capabilities means that the specific accuracy figures and behavioral patterns observed may not hold for future model versions. The experimental design and evaluation method, however, are transferable.
The authors designed both the agentic solutions and the evaluation, which introduces

a potential bias in the selection of scenarios and the design of assessment questions. To mitigate this, all assessment questions were verified against the actual log data, correct answers were validated independently, and participant results are reported without selective omission.

7.4 Construct Validity

The NASA-TLX instrument is validated for measuring subjective workload, but the combination used here, five scales without pairwise weighting and with Physical Demand excluded, has not been validated as a bundle. Dropping a scale is common in knowledge-worker studies, and unit-weighted scales are a recognized simplification of the raw TLX, so each change has precedent even if the full combination does not. Individual interpretations of the scales also vary between participants. P5 rated time pressure and frustration at 2 in both conditions, while the other scales varied substantially, suggesting that different participants used the scales differently. With only six participants, this variation cannot be corrected for.

The multiple-choice questions were designed to test diagnostic reasoning at six stages (identification, timeline, status, data loss, scale, assessment). Each question was verified against the actual log data to ensure the correct answer was clearly derivable from the evidence. A pilot session with an experienced engineer led to revisions of questions that required system knowledge beyond the logs. However, the questions test the ability to select a correct answer from four options, which is a different skill from open-ended diagnostic reasoning in real support work.

The agent accuracy evaluation used the same questions as the engineer experiment, enabling direct comparison. However, batch mode (all questions in a single prompt) gives the agent an advantage that engineers do not have. The agent can plan its analysis around all six questions simultaneously, while engineers answer one question at a time without knowing future questions.

The Case 2 agent-accuracy metrics exhibit several limitations in construct validity. Source Recovery and Diagnostic Convergence return 1.0 by convention when the raw-run reference produces no parsable evidence references. Some scores reported as perfect are therefore vacuously perfect, and these contribute to the reported means. Dispatch Efficiency and Redundancy are defined as complementary ratios that sum to one by construction. They contribute a single independent measurement rather than two, and reporting both counts as a single independent metric.

Diagnostic Convergence uses a flat, keyword-based category classifier with approximately 60 hand-written regular expressions rather than the hierarchical taxonomy implied by the Lim et al. citation. Inter-rater reliability on the classifier was not measured. Actionability is scored based on regex template compliance across eight items: any numbered list counts as ordered steps, and any mention of a documentation system counts as a documentation reference. The high mean (0.92), therefore, reflects structural scaffolding in the agent's output rather than content quality. Source Recovery counts trouble-report identifiers, DTS keys, and case IDs but excludes URL and EKSS references that the extractor also produces, restricting the metric's scope to a subset of the evidence types the agent cites.

The masking layer in the CSTS proxy strips resolution-bearing fields from tool-

retrieved payloads but cannot prevent the agent from reproducing customer identifiers it encountered earlier in the session through its own generated text. This is a limitation of the masking design rather than the evaluation, but it means that some masked runs may contain information intended to be withheld.

The open-ended evaluation scored agent responses against checklists developed by a single author, and scoring was performed by the same author. This introduces a single-rater bias. A different researcher might include different findings or interpret edge cases differently, and no inter-rater agreement statistic is available. To mitigate this, findings were extracted from trouble report resolutions written by the engineers who resolved the original issues, not invented by the author, and every finding is cross-cited to three sources, namely the TRTS resolution, the exact log-file line or record, and the agent's response, so that every verdict is independently verifiable by any reader. Scoring is strict binary (Found, Missed, or Incorrect) with no weights or partial credit. Findings requiring code-level knowledge or TRTS metadata were excluded up front. The number of findings varies across trouble reports (5 to 15), reflecting different fault complexities, so individual per-TR percentages are not directly comparable. The aggregate score across all 12 trouble reports ($87/107 = 81\%$) provides a more stable measure.

Chapter 8

Conclusions and Future Work

8.1 Conclusions

This thesis investigated how agentic AI workflows affect the performance and cognitive load of support engineers (RQ1) and how accurately the solutions retrieve relevant information (RQ2).

Regarding RQ1, in the log analysis case, AI assistance was associated with higher diagnostic accuracy for four of six participants, while one maintained the same score and one scored lower. The median accuracy rose from 2.5 to 4.0 out of 6, an improvement of $+1.5$ with individual deltas ranging from -2 to $+4$, with gains concentrated on questions requiring timeline analysis and pattern identification in large log files. Median session time was 4 minutes 17 seconds longer in the AI-assisted condition, a difference that must be interpreted cautiously given the small sample and technical disruptions affecting four of six sessions. All five NASA-TLX workload scales decreased to the median level, although the relationship between perceived workload and accuracy was inconsistent across participants.

AI assistance also shifted the nature of errors from information-retrieval failures to information-interpretation and prompting failures. The majority of wrong answers in AI-assisted rounds occurred when the AI provided correct information that the engineer misinterpreted or when the engineer prompted the AI poorly. Prompt quantity alone did not determine the outcome. Participants who used the AI as an interactive investigation tool tended to achieve higher scores. Two participants exhibited the *false ease* pattern, where perceived workload dropped substantially while accuracy did not improve. In safety-critical diagnostic work, a tool that reduces perceived effort without improving outcomes represents a usability risk that future interaction design should address. This is an exploratory observation based on two data points and warrants investigation with larger samples before it can be considered a generalizable finding.

Regarding RQ2, in the log analysis case, the agent achieved 100% accuracy on structured multiple-choice questions when given the full question set and 81% (87 of 107 findings) on open-ended diagnostic analysis across 12 trouble reports spanning 12 distinct fault types. Four of 12 trouble reports scored 100%. The remaining eight contained missed findings that cluster into six recurring failure modes, analyzed in Section 6.2.1.1. The agent is a strong evidence retriever and a weak evaluative reasoner. It reliably surfaces what the logs say but less reliably evaluates whether what the logs say is appropriate for what they describe. The gap between agent-alone accuracy and engineer-with-AI accuracy (median 4/6) suggests a hypothesis for future

research: the primary limitation in AI-assisted log analysis may be the human-tool interaction rather than the AI's analytical capability. This comparison is not direct, as the agent received all questions simultaneously while engineers answered sequentially, but the pattern is consistent with the wrong-answer analysis, which shows that most errors were on the human side.

In the information search case, AI assistance raised the median accuracy from 4 out of 6 to 5 out of 6 for five of six participants, with one participant scoring lower. All five NASA-TLX workload scales decreased at the median level, and the effect was larger than in the log analysis case, with one participant reporting the largest individual workload reduction observed in either case. Session time was longer in the AI condition, but the difference was largely attributable to the 10 to 12 minutes of autonomous investigation that executed before the engineer began answering. The agent scored perfectly on the same questions in both solo and multi-agent batch mode (Table 5.21), indicating that the information needed to answer correctly was present in the agent's output. As in the log analysis case, the gap between agent and engineer accuracy was driven more by the interaction than by the AI's analytical capability. The four wrong answers in the AI condition split into two distinct failure modes. On Ticket-C, engineers anchored on their own hypotheses despite the agent presenting the correct answer. On Ticket-D, engineers followed the agent's top-ranked hypothesis without challenging it, even when the agent's own text flagged missing evidence. Five of six participants reported needing more information in the manual round, yet none raised the same concern in the AI round, despite both rounds operating on the same data. While the immediate effect is positive, the observed pattern of perceived sufficiency suggests a long-term risk of deskilling if engineers rely on AI-structured syntheses without maintaining their manual investigation skills.

Regarding RQ2, the information search agent produced a reproducible investigative process across 90 runs on 30 cases (operational metrics ≥ 0.89, three-round contract satisfied on 89 of 90 runs) but recovered only about half of the evidence references on average (Source Recovery 0.47, standard deviation 0.25). The evaluation measures self-consistency under information loss rather than classification accuracy against a ground truth, so the Case 2 figures are not directly comparable to the Case 1 finding rate. The principal observation is the split between stable process and variable evidence recall, which implies that a single invocation of the agent is not a reliable indicator of what it will find on a given case.

The agents on their own were reliable evidence retrievers, scoring 100 % on structured multiple-choice, 81 % on open-ended log analysis, and compliance with the three-round dispatch contract on 89 of 90 information-search runs. The engineers working with those same agents did not reach that ceiling. In both cases, most wrong answers in the AI condition followed correct or plausibly correct AI output that the engineer did not translate, did not challenge, or did not consult. The gap between the two measurements is therefore observational rather than directional. It sits in how the engineer reads, weights, and interrogates what the agent returns rather than in the agent's ability to find evidence on a real support case.

This observation returns to the motivating problem of the thesis. The fragmentation of diagnostic data across disconnected tools slows the work of support engineers on the faults that telecommunications networks depend on. The two solutions reduced that fragmentation in the observed results. Median diagnostic accuracy rose in both

cases, all five NASA-TLX workload scales fell at the median, and the information search agent dispatched a structured investigation across six systems where the engineer would otherwise have done so by hand. Reducing fragmentation, however, did not close the accuracy gap on its own. The engineer remained the final reader and judge of what the agent produced, and the design of that interaction is as important as the capability of the underlying model in shaping the practical impact of agentic AI on support engineering.

8.2 Future Work

The evaluation was conducted with six participants from a single support team, with a balanced group assignment (three in Group A, three in Group B). Increasing the sample size with balanced groups and replicating with teams from different products or companies would strengthen the findings.

The variation in AI usage patterns and the dominance of interaction-design limitations (11 of 13 wrong answers in Case 1 were attributable to engineer-tool interaction rather than AI failure) suggest that interaction design and engineer skills matter as much as model capability. Future work could investigate how different interface designs and structured onboarding narrow the gap between agent alone and engineer with AI accuracy.

Several missed findings in the open-ended evaluation involved log patterns that would map to already-known software bugs. Connecting the log analysis agent to external knowledge sources such as bug databases could help bridge this gap. A broader direction is context engineering through structured intermediate representations. A per-case knowledge graph linking trouble reports, DTS keys, and components could support the agent's reasoning in a way that does not depend on what a given run happens to retrieve. Graph-backed retrieval has been explored for related tasks [37] but not with MCP-governed tool access. A complementary direction is modular agent skill definitions, such as Anthropic's Skills [43], which package task-specific instructions, procedures, and tool bundles in a loadable format and could let the log analysis and information search agents share reusable diagnostic patterns across cases.

The false-ease risk, in which AI reduces perceived difficulty without improving accuracy, warrants further investigation with larger samples to determine whether it is an individual anomaly or a systematic pattern.

Longitudinal studies could track how AI usage patterns evolve as engineers gain experience with the tool, showing whether the adoption barriers observed in this study diminish over time.

The evidence-recall variability in the information search agent (Source Recovery standard deviation up to 0.53 on individual cases) motivates self-consistency and ensemble strategies, where multiple runs on the same case are reconciled before presentation.

A direct comparison of the solo and multi-agent architectures on the full 30-case corpus, out of scope in this study, would also help characterize the trade-off between investigative breadth and runtime cost.

The automation bias observed in the information search case, where engineers

stopped reporting missing information when the AI provided a structured synthesis of the same data, warrants further study. One concrete deployment question is whether this pattern changes engineer-to-customer request rates in real support work, which would be measurable by comparing request volumes before and after agent introduction. Whether the effect persists as engineers gain experience, and whether interface features that flag missing evidence can counteract it, would inform future system design.

The problem-identification phase produced four candidate cases (Table 4.3), of which two were implemented in this thesis. The remaining two are natural directions for future work. Case 3, knowledge sharing, would target the experience gap that interviews identified as the second-ranked bottleneck, an agent that captures the reasoning trace of an experienced engineer resolving a trouble report, distils it into a reusable pattern, and retrieves that pattern on future similar cases could address the knowledge-distribution problem that the retrieval-augmented agents in this thesis do not. Case 4, customer connectivity, would target the customer-communication bottleneck and connects back to the automation-bias finding, an agent that helps the engineer structure a clarification request to the customer, rather than synthesising the existing ticket, would improve the quality of requests when they are warranted rather than reducing their number through perceived sufficiency. Three further interview bottlenecks that did not become candidate cases (access to the product on the customer side, ticket writing, and insufficient test-scenario coverage) are outside the read-only retrieval scope adopted in this thesis but remain open directions for agentic support in telecommunications engineering.

Finally, the figures in this thesis were obtained on Kiro CLI 1.27.2 with Claude Opus 4.6 in April 2026. Given the pace of model and MCP evolution, continuous re-evaluation against the same corpora would separate design-level from model-level limitations. A related direction is extending the architecture beyond read-only retrieval into tiered policy-governed actions [17], while continuing to preserve engineer authority over diagnostic decisions.

References

[1] M. Arnold, M. Goldschmitt, and T. Rigotti, "Dealing with information overload: A comprehensive review," *Frontiers in Psychology*, vol. 14, p. 1122200, 2023.

[2] C. Pei, Z. Wang, F. Liu, Z. Li, Y. Liu, X. He, R. Kang, T. Zhang, J. Chen, J. Li, G. Xie, and D. Pei, "Flow-of-Action: Sop enhanced llm-based multi-agent system for root cause analysis," in *Companion Proceedings of the ACM Web Conference 2025 (WWW Companion '25)*, 2025.

[3] X. Hou, Y. Zhao, S. Wang, and H. Wang, "Model context protocol (MCP): Landscape, security threats, and future research directions," *ACM Transactions on Software Engineering and Methodology*, 2025. Accepted. Preprint: arXiv:2503.23278.

[4] D. Feng, B. Yun, and A. Y. Wang, "From junior to senior: Allocating agency and navigating professional growth in agentic AI-mediated software engineering," in *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26)*, (Barcelona, Spain), ACM, 2026.

[5] K. Wu, Q. Yu, M. Mei, R. Liu, J. Wang, K. Zhang, Y. Bao, W. Wan, R. Ye, B. He, J. Liao, Y. Du, Z. Yang, K. Liu, L. Huang, Z. Song, Y. Gao, F. Tan, J. Yang, and N. Gu, "TN-AutoRCA: Benchmark construction and agentic framework for self-improving alarm-based root cause analysis in telecommunication networks," *arXiv preprint arXiv:2507.18190*, 2025.

[6] S. G. Hart and L. E. Staveland, "Development of NASA-TLX (Task Load Index): Results of empirical and theoretical research," in *Advances in Psychology*, vol. 52, pp. 139–183, North-Holland, 1988.

[7] Y. Tang and T. Runkler, "LLM-based agentic systems for software engineering: Challenges and opportunities," *arXiv preprint arXiv:2601.09822*, 2026.

[8] P. Pirolli and S. Card, "Information foraging," *Psychological Review*, vol. 106, no. 4, pp. 643–675, 1999.

[9] G. Klein, "Naturalistic decision making," *Human Factors*, vol. 50, no. 3, pp. 456–460, 2008.

[10] J. Lee, Y. Jeong, T. Han, and T. Lee, "LogRESP-Agent: A recursive AI framework for context-aware log anomaly detection and TTP analysis," *Applied Sciences*, vol. 15, no. 13, p. 7237, 2025.

[11] W. Xu, J. Luo, T. Huang, K. Sui, J. Geng, Q. Ma, I. Akasaka, X. Shi, J. Tang, and P. Cai, "LogSage: An LLM-based framework for CI/CD failure detection

and remediation with industrial validation," *arXiv preprint arXiv:2506.03691*, 2025.

[12] D. Deshpande, V. Gangal, H. Mehta, J. Krishnan, A. Kannappan, and R. Qian, "TRAIL: Trace reasoning and agentic issue localization," *arXiv preprint arXiv:2505.08638*, 2025.

[13] D. Lodha, M. Panchal, and S. G. Kulkarni, "MCP-Diag: A deterministic, protocol-driven architecture for AI-native network diagnostics," in *Proceedings of the 18th International Conference on Communication Systems and Networks (COMSNETS 2026), Graduate Forum*, 2026.

[14] C. C. Liao, D. Liao, and S. S. Gadiraju, "AgentMaster: A multi-agent conversational framework using A2A and MCP protocols for multimodal information retrieval and analysis," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP 2025)*, 2025.

[15] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large language model connected with massive APIs," in *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pp. 126544–126565, 2024.

[16] C. Bandi, B. Hertzberg, G. Boo, T. Polakam, J. Da, S. Hassaan, M. Sharma, A. Park, E. Hernandez, D. Rambado, I. Salazar, R. Cruz, C. Rane, B. Levin, B. Kenstler, and B. Liu, "MCP-Atlas: A large-scale benchmark for tool-use competency with real MCP servers," *arXiv preprint arXiv:2602.00933*, 2026.

[17] S. Ahuja, N. Kordjazi, E. Yortucboylu, V. Kapoor, M. Dundua, Y. Li, D. Ho, V. Padala, J. Whitted, and R. Steinert, "VIGIL: Towards edge-extended agentic AI for enterprise IT support," *arXiv preprint arXiv:2603.16110*, 2026.

[18] N. Tacheny, "Agentic diagnostic reasoning over telecom and datacenter infrastructure," *arXiv preprint arXiv:2601.07342*, 2026.

[19] P. Offermann, O. Levina, M. Schönherr, and U. Bub, "Outline of a design science research process," in *Proceedings of the 4th International Conference on Design Science Research in Information Systems and Technology (DESRIST 2009)*, vol. 5951 of *LNCS*, pp. 1–11, Springer, 2009.

[20] L. Silva, *On Identifying Technical Debt Using Bug Reports in Practice*. Licentiate dissertation, Blekinge Institute of Technology, Karlskrona, Sweden, 2023.

[21] S. M. Blackburn, A. Diwan, M. Hauswirth, P. F. Sweeney, J. N. Amaral, V. Babka, W. Binder, T. Brecht, L. Bulej, L. Eeckhout, S. Fischmeister, D. Frampton, R. Garner, A. Georges, L. J. Hendren, M. Hind, A. L. Hosking, R. E. Jones, T. Kalibera, P. Moret, N. Nystrom, V. Pankratius, and P. Tuma, "Can you trust your experimental results?," Tech. Rep. 1, Evaluate Collaboratory, Feb. 2012.

[22] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Berlin, Heidelberg: Springer, 2 ed., 2024. Crossover / paired comparison design: §9.5, pp. 122–123; statistical analysis: ch. 11, pp. 183–184.

[23] J. Mayfield, E. Yang, D. Lawrie, S. MacAvaney, P. McNamee, D. W. Oard, L. Soldaini, I. Soboroff, O. Weller, E. Kayi, K. Sanders, M. Mason, and N. Hibbler, "On the evaluation of machine-generated reports," in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '24)*, 2024.

[24] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "Swe-bench: Can language models resolve real-world github issues?," in *Proceedings of the 12th International Conference on Learning Representations (ICLR 2024)*, 2024.

[25] S. Min, K. Krishna, X. Lyu, M. Lewis, W.-t. Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, and H. Hajishirzi, "FActScore: Fine-grained atomic evaluation of factual precision in long form text generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)*, 2023.

[26] S. Lim, G. Kim, H. Lee, W. Han, J. Seo, J. Yoo, and E. Yang, "H-DDx: A hierarchical evaluation framework for differential diagnosis," in *NeurIPS 2025 Workshop on GenAI for Health (Second Workshop)*, 2025. arXiv:2510.03700.

[27] S. Cui, R. Krishna, S. Jha, and R. K. Iyer, "Agentic structured graph traversal for root cause analysis of code-related incidents in cloud applications," 2025. arXiv:2512.22113.

[28] S. Mehta, "Beyond accuracy: A multi-dimensional framework for evaluating enterprise agentic AI systems," 2025. arXiv:2511.14136.

[29] H. Wei, Z. Zhang, S. He, T. Xia, S. Pan, and F. Liu, "PlanGenLLMs: A modern survey of LLM planning capabilities," 2025. arXiv:2502.11221.

[30] P. He, Z. Dai, B. He, H. Liu, X. Tang, H. Lu, J. Li, J. Ding, S. Mukherjee, S. Wang, Y. Xing, J. Tang, and B. Dumoulin, "TRAJECT-Bench: A trajectory-aware benchmark for evaluating agentic tool use," 2025. arXiv:2510.04550.

[31] S. Xia, X. Li, Y. Liu, T. Wu, and P. Liu, "ReasonEval: Evaluating reasoning quality of language models," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI 2025)*, 2025.

[32] J. Shao, Y. Lin, M. P. Lohani, Y. Miao, and B. Luo, "Do LLM agents know how to ground, recover, and assess? A benchmark for epistemic competence in information-seeking agents," 2025. arXiv:2509.22391.

[33] K. J. Meimandi, G. R. Kim, G. Aránguiz-Dias, L. Saadeddin, A. Griffith, and M. J. Kochenderfer, "The measurement imbalance in agentic AI evaluation undermines industry productivity claims," 2025. arXiv:2506.02064.

[34] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation)." Official Journal of the European Union, L 119, 1–88, 2016.

[35] Swedish Research Council, "Good research practice." Swedish Research Council, 2017.

[36] Swedish Parliament, "Lag (2003:460) om etikprövning av forskning som avser människor [The Ethical Review Act]." Svensk författningssamling, SFS 2003:460, 2003.

[37] Y. Lu, W. Wu, X. Zhao, R. Peng, and J. Wang, "KARMA: Leveraging multi-agent LLMs for automated knowledge graph enrichment," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2502.06472.

[38] R. S. Nickerson, "Confirmation bias: A ubiquitous phenomenon in many guises," *Review of General Psychology*, vol. 2, no. 2, pp. 175–220, 1998.

[39] R. Parasuraman and V. Riley, "Humans and automation: Use, misuse, disuse, abuse," *Human Factors*, vol. 39, no. 2, pp. 230–253, 1997.

[40] P. Runeson and M. Höst, "Guidelines for conducting and reporting case study research in software engineering," *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009.

[41] B. Kitchenham, J. Fry, and S. Linkman, "The case against cross-over designs in software engineering," in *Proceedings of the 11th International Workshop on Software Technology and Engineering Practice (STEP)*, pp. 65–67, 2003.

[42] J. Frattini, D. Fucci, and S. Vegas, "Crossover designs in software engineering experiments: Review of the state of analysis," in *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM '24)*, (Barcelona, Spain), ACM, 2024.

[43] Anthropic, "Agent skills." `https://docs.claude.com/en/docs/agents-and-tools/agent-skills/overview`, 2025. Accessed: 2026-04-27.

Appendix A

Interview Guide

The following questions were used as a guide during the semi-structured interviews with support engineers. The interviews were conducted in Swedish and followed a conversational format, allowing follow-up questions based on the participant's responses.

Phase 1: Triage and Information Gathering

1. What is the first thing you do when a new ticket arrives?
2. How do you make sure you have enough information to start investigating? What do you do when the ticket description is incomplete?

Phase 2: Troubleshooting and Problem Solving

3. Walk me through what happens when a standard solution does not work. What are your steps for a deeper investigation?
4. Which contacts do you rely on, and at what point do you involve them?
5. Have you ever reached a point where a fix was not possible? What did you do next?

Phase 3: Bottlenecks and Efficiency

6. Can you describe a recent ticket that took a long time to resolve? Looking back, what slowed it down and what could have been done differently?
7. What do you see as the biggest bottleneck in your current workflow?

Phase 4: Tools and Future Improvements

8. What tools do you use on a daily basis, and for what tasks?
9. Do you see opportunities for automation in your current workflow? If so, which tasks would you prioritize?

Appendix B

Assessment Questions

The following multiple-choice questions were used in the log analysis experiment. Each participant answered all six questions for both scenarios. The correct answer is marked with an asterisk (*). Internal product and component names have been replaced with generic terms.

Ticket-A: Provisioning System Exceptions

1. **What happened:** The transaction log shows both successful and failed operations. What pattern do the failures follow?

   A) It depends on the subscriber. The ones that fail always fail, and the ones that succeed always succeed. *

   B) It depends on timing. Failures come in bursts with successful periods in between.

   C) It depends on which client sends the request. One client gets errors while the other does not.

   D) It depends on which application pod handles the request. One pod returns errors while the other works fine.

2. **Impact:** The application log shows continuous exceptions for failed lookups. Do they affect the processing of successful requests?

   A) Yes. The exceptions cause delays that slow down all requests.

   B) Yes. The system pauses all processing while it handles each exception.

   C) No. Successful requests continue to complete normally alongside the failures. *

   D) No. The system isolates failures by routing them to a separate component.

3. **Caller response:** When a lookup fails, what error does the client get back?

   A) It identifies the specific reason. The client knows what went wrong. *

   B) It gives a generic timeout error. The client cannot tell whether the subscriber exists.

   C) It reports success. The client does not know the lookup failed.

D) It returns an error that suggests a temporary problem. The client may retry expecting it to work later.

4. **Root cause:** The error response to the client is accurate, the subscriber does not exist. But the logs are rotating within minutes. What is causing the log volume?

   A) Each failed lookup is retried several times before the system gives up.

   B) The code throws a full exception for a normal error instead of handling it quietly. *

   C) The downstream system is unstable and the reconnection attempts fill the logs.

   D) The logging level is misconfigured, capturing output that should be hidden.

5. **Scale:** How many unique subscribers are failing on directory lookups in the transaction log?

   A) Fewer than 20.

   B) About 75.

   C) About 125. *

   D) Over 500.

6. **Assessment:** What is your assessment, and what should happen next?

   A) The subscriber data is out of sync. The customer needs to reload their subscriber database.

   B) The exception handling is broken. Development needs to fix how the system logs this type of error. *

   C) The downstream system returns a bad response. Development needs to fix the parser.

   D) The log volume is an operational issue. The customer should increase the log rotation size.

Ticket-B: Platform Upgrade Disruption

1. **What happened:** Look at the test case log. What did the test framework do that caused the disruption, and which component was affected?

   A) The message bus pods were restarted by a helm upgrade that changed their memory allocation. *

   B) The coordination service pods were restarted by a helm upgrade that changed their memory allocation.

   C) The document database pods were restarted by a helm upgrade that changed their memory allocation.

   D) The ingress controller pods were restarted by a helm upgrade that changed their memory allocation.

2. **Timeline:** The application log contains two different categories of errors. Did they start at the same time?

   A) Both started at the same time when the helm upgrade ran.
   B) One was already happening before the upgrade. The other started when the upgrade ran. *
   C) Both started after the upgrade, but about 5 minutes apart.
   D) Both were already happening before the upgrade. Neither is related to it.

3. **Current status:** Look at the end of the application log. Has the system recovered?

   A) Both categories stopped. The system fully recovered.
   B) Neither recovered. Both are still ongoing at the same rate.
   C) One category recovered, but the other is still ongoing. *
   D) Neither fully stopped, but both are less frequent than before.

4. **Data loss:** Data failed to reach the message bus during the disruption. What was it?

   A) Audit logs of completed operations and alarm clear notifications. *
   B) Incoming provisioning requests and alarm raise notifications.
   C) Only audit logs of completed operations. No other data was affected.
   D) Only alarm notifications. No other data was affected.

5. **Scale:** How does the volume of log output compare to the actual duration of the disruption?

   A) A few thousand warnings over about 30 minutes.
   B) About ten thousand warnings over about 15 minutes.
   C) Tens of thousands of warnings in about 5 minutes. *
   D) Tens of thousands of warnings over about 30 minutes.

6. **Assessment:** What is your assessment of the two issues, and what should happen next?

   A) Both only occurred during the upgrade and have stopped. No action needed.
   B) The warning volume is a product defect. The connectivity errors are unrelated and need separate investigation. *
   C) The connectivity errors caused the warning volume. Fixing the connectivity will fix both.
   D) Neither issue is related to the upgrade. Both are pre-existing configuration problems.

Case 2: Information Search

The following multiple-choice questions were used in the information search experiment. Each participant answered all six questions for both scenarios. The correct answer is marked with an asterisk (*). Internal product and component names have been replaced with generic terms.

Ticket-C: Monitoring Heartbeat Flapping

1. **Problem identification:** Based on the case information, what characterizes the heartbeat failure the customer is experiencing?

   A) The system nodes are crashing and restarting, causing the heartbeat to drop during each restart cycle.

   B) The network management system shows the heartbeat state for both nodes flapping between IN_SERVICE and OUT_OF_SERVICE, but no alarm or anomaly is visible from the product side itself. *

   C) The heartbeat is permanently showing OUT_OF_SERVICE for both nodes because the monitoring agent pods are stuck in a restart loop.

   D) The heartbeat failure only affects one node and is caused by a network connectivity issue between the product and the network management system.

2. **Root cause:** Based on your investigation, what is the most likely root cause of the heartbeat flapping?

   A) The monitoring agent stops sending heartbeat traps after configuration changes, a known platform defect.

   B) Both nodes in the active-standby pair are configured with the same SNMP engine ID, so the network management system cannot distinguish between them and alternates the heartbeat state as traps arrive from different nodes. *

   C) The provisioning interface is disconnecting intermittently, which causes the monitoring agent to stop sending heartbeat traps during disconnection periods.

   D) An NTP time synchronisation drift between the nodes and the network management system causes heartbeat traps to arrive outside the expected time window.

3. **Validation:** What validation step would you recommend to confirm the root cause before applying a fix?

   A) Capture a TCP dump between the product and the network management system to verify that heartbeat traps are being sent at the correct interval.

   B) Restart the monitoring agent pods on both nodes simultaneously and check if the flapping stops.

C) Scale down the monitoring pod on the standby node only, then monitor whether the heartbeat alarm is suppressed on the active node. *

D) Temporarily switch from SNMPv3 to SNMPv2c to rule out engine ID authentication issues.

4. **Resolution:** What fix would you recommend for this issue?

A) Upgrade the monitoring agent to a newer version that generates random engine IDs automatically on startup.

B) Change the engine ID in the monitoring agent configuration to a unique value on the standby node, then restart the monitoring agent deployment. *

C) Reconfigure the network management system to use IP-based node identification instead of SNMP engine ID-based identification.

D) Add a firewall rule on the standby node to block outgoing SNMP heartbeat traps, so only the active node sends them.

5. **Evidence cross-referencing:** Based on cross-customer evidence, how widespread is this issue?

A) This is a unique issue specific to this customer's network management configuration.

B) This is a widespread issue affecting multiple customers across multiple product versions, all caused by duplicate SNMP engine IDs in active-standby deployments where the default or example value was used on both nodes. *

C) This issue only affects one specific product version due to a regression in the monitoring agent introduced in that version.

D) This issue only occurs with one specific network management system.

6. **Assessment:** What would your overall assessment and recommendation be for this case?

A) This is a network management system issue. The product is working correctly.

B) This is a configuration issue where both nodes share the same SNMP engine ID. The fix is to set a unique engine ID on the standby node. This is a widespread issue, with a permanent platform fix in newer versions that auto-generates unique IDs. *

C) This is a platform defect in the monitoring agent that requires an emergency patch.

D) This is a known limitation of the active-standby architecture. Heartbeat flapping is expected behavior during failover events.

Ticket-D: Stuck Asynchronous Requests

1. **Problem identification:** Based on the case information, what is happening with the customer's provisioning requests?

   A) Asynchronous provisioning requests are stuck in the request handler. They were accepted but never completed or returned a result. *

   B) Synchronous provisioning requests are timing out at the northbound interface because the processing engine pods are stuck in a restart loop.

   C) Provisioning requests are completing but the northbound callback notification is failing, so the upstream system thinks they are stuck.

   D) Provisioning requests are queued in the message bus but the consumer group has stalled due to a partition rebalance.

2. **Root cause:** Based on your investigation, what is the most likely root cause of the stuck requests?

   A) A database disk space issue caused the pending-jobs table to become read-only, preventing the request handler from updating request status.

   B) A defect in the request handler's fetch task where the time-compensation window is too small. Requests that arrive during the gap between fetch cycles are never picked up for processing. *

   C) A race condition in the request handler where concurrent requests for the same subscriber cause a deadlock.

   D) A memory leak in the request handler's notification component that gradually exhausts the pod's heap space.

3. **Workaround:** What procedure would you recommend to recover the stuck requests?

   A) Restart the request handler pods. The fetch task will re-scan the pending-jobs table on startup.

   B) Increase the fetch task time-compensation window via a deployment-values override, then wait for the stuck requests to be picked up.

   C) Stop incoming traffic, allow executing requests to complete, clean the remaining pending cases from the database, restart the pods, then re-enable traffic. *

   D) Scale down the request handler to zero replicas, truncate the message-bus topic to clear the backlog, then scale back up.

4. **Evidence cross-referencing:** Based on cross-customer evidence, how widespread is this issue?

   A) This is a rare issue seen only at two customer sites on one specific product version.

   B) This is a widespread issue affecting multiple customers across multiple product versions, all linked to the same request handler defect family. *

C) This issue only occurs on cloud-native deployments. Virtual installations are not affected.

D) This issue only occurs with one specific upstream system integration.

5. **Assessment:** What would your overall assessment and recommendation be for this case?

A) This is a database performance issue. The customer should increase disk throughput.

B) This is a confirmed product defect in the request handler's fetch task timing. The cleanup workaround recovers stuck requests, but the permanent fix requires upgrading to a newer version where the time-compensation window was increased. *

C) This is an operational issue caused by the customer sending too many concurrent requests.

D) This is a known limitation of the asynchronous processing architecture. The customer should switch to synchronous mode.

6. **Defect tracking:** Which trouble report documents this defect and what is the fix?

A) A trouble report for requests stuck during cluster startup, fixed by adding a startup delay.

B) A trouble report for the fetch task time-compensation window, increased from the original value to a larger window, permanently fixed in a newer product version. *

C) A trouble report for requests stuck during node reboots, fixed by persisting the fetch pointer across restarts.

D) A trouble report for queue corruption during database compaction, fixed by adding checksum validation.

Appendix C

Per-Case Stability Data

Table C.1: Per-case stability of the two non-operational metrics across 3 runs per case. Operational metrics (actionability, dispatch efficiency, redundancy) showed $\sigma \leq 0.14$ on every case and are omitted here. Customer-Ticket-M2 and Customer-Ticket-H9 are the two tickets used in the engineer experiment (Ticket-D and Ticket-C, respectively).

Case	Diff.	SrcRec ($\mu \pm \sigma$)	DiagConv ($\mu \pm \sigma$)
Customer-Ticket-L1	low	0.36 ± 0.08	1.00 ± 0.00
Customer-Ticket-L2	low	0.69 ± 0.53	0.78 ± 0.38
Customer-Ticket-L3	low	0.55 ± 0.13	0.56 ± 0.19
Customer-Ticket-L4	low	0.78 ± 0.19	0.67 ± 0.00
Customer-Ticket-L5	low	0.82 ± 0.17	0.67 ± 0.00
Customer-Ticket-L6	low	0.21 ± 0.07	0.44 ± 0.38
Customer-Ticket-L7	low	0.35 ± 0.05	0.44 ± 0.38
Customer-Ticket-L8	low	0.53 ± 0.15	0.56 ± 0.19
Customer-Ticket-L9	low	0.39 ± 0.10	0.67 ± 0.00
Customer-Ticket-L10	low	0.43 ± 0.22	0.89 ± 0.19
Customer-Ticket-M1	medium	0.43 ± 0.13	0.78 ± 0.19
Customer-Ticket-M2	medium	0.51 ± 0.14	0.89 ± 0.19
Customer-Ticket-M3	medium	0.85 ± 0.06	0.89 ± 0.19
Customer-Ticket-M4	medium	0.78 ± 0.04	1.00 ± 0.00
Customer-Ticket-M5	medium	0.23 ± 0.05	0.56 ± 0.38
Customer-Ticket-M6	medium	0.16 ± 0.10	0.44 ± 0.19
Customer-Ticket-M7	medium	0.45 ± 0.05	0.67 ± 0.33
Customer-Ticket-M8	medium	0.18 ± 0.03	0.44 ± 0.19
Customer-Ticket-M9	medium	0.19 ± 0.14	0.67 ± 0.33
Customer-Ticket-M10	medium	0.16 ± 0.05	0.56 ± 0.38
Customer-Ticket-H1	high	0.40 ± 0.07	0.89 ± 0.19
Customer-Ticket-H2	high	0.61 ± 0.11	0.78 ± 0.19
Customer-Ticket-H3	high	0.19 ± 0.02	0.44 ± 0.19
Customer-Ticket-H4	high	0.44 ± 0.17	0.89 ± 0.19
Customer-Ticket-H5	high	0.49 ± 0.16	1.00 ± 0.00
Customer-Ticket-H6	high	0.70 ± 0.15	0.89 ± 0.19
Customer-Ticket-H7	high	1.00 ± 0.00	1.00 ± 0.00
Customer-Ticket-H8	high	0.49 ± 0.04	0.78 ± 0.19
Customer-Ticket-H9	high	0.31 ± 0.04	0.56 ± 0.19
Customer-Ticket-H10	high	0.43 ± 0.05	1.00 ± 0.00

Appendix D

Implementation Details

Kiro is distributed as a command-line tool. To make it callable from a web frontend, the CLI is wrapped behind a thin HTTP API implemented in Flask. The API exposes request-response and streaming endpoints. The streaming endpoint parses Kiro's stdout into Server-Sent Events so that intermediate reasoning, sub-agent dispatches, and tool calls can be rendered live in the UI. The Flask service itself is stateless. Each request spawns a fresh `kiro-cli` subprocess, and chat transcripts are persisted by `kiro-cli` on a mounted volume rather than by the service. Both cases share the same deployed backend and select their behavior through a request body that names the agent and working directory.

A separate frontend was built for each case using Next.js. Next.js was chosen because its server-rendered React components can call the Kiro API directly from server-side code, keeping credentials and system prompts out of the browser. Each frontend owns a client-issued session identifier that the backend uses to isolate concurrent conversations on the filesystem. The per-case system prompts are loaded by the backend from the case's own directory, so the two cases share the Kiro runtime without interfering with each other.

The top-level topology is therefore three tiers, illustrated in Figure D.1: two Next.js frontends (one per case), a single shared Flask deployment running under four `gunicorn` workers, and the `kiro-cli` subprocess that each request spawns. The experiments reported in this thesis were run against a local Docker Compose stack. A Kubernetes deployment path (with a persistent volume claim for `kiro-cli` state and a LoadBalancer service) exists in the repository but was not exercised in the evaluation. Authentication, rate limiting, and application monitoring were out of scope. The goal of the artefacts is to support the controlled experiments described in Section 4.3.3, not to serve engineers at steady state.

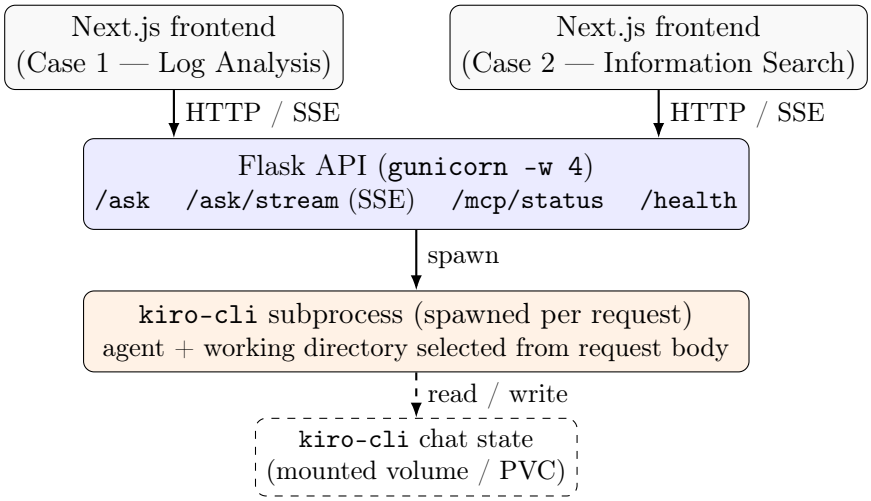


Figure D.1: Top-level application architecture. Two Next.js frontends target the same Flask deployment, which selects each case's agent and working directory from the request body and spawns a fresh `kiro-cli` subprocess per request. The Flask service is stateless; chat state is persisted by `kiro-cli` on a mounted volume.

Appendix E

Result Visualizations

This appendix presents visual summaries of the key results reported in Chapter 5. The underlying data are available in the tables of the main text.

Case 1: Log Analysis

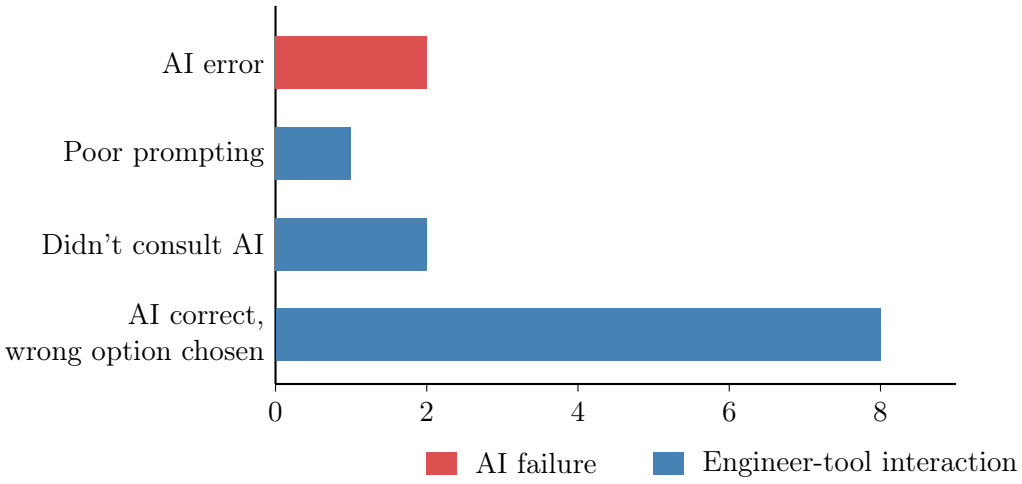


Figure E.1: Case 1: Error classification for the 13 wrong answers in the AI-assisted condition. 11 of 13 errors were attributable to engineer-tool interaction rather than AI failure.

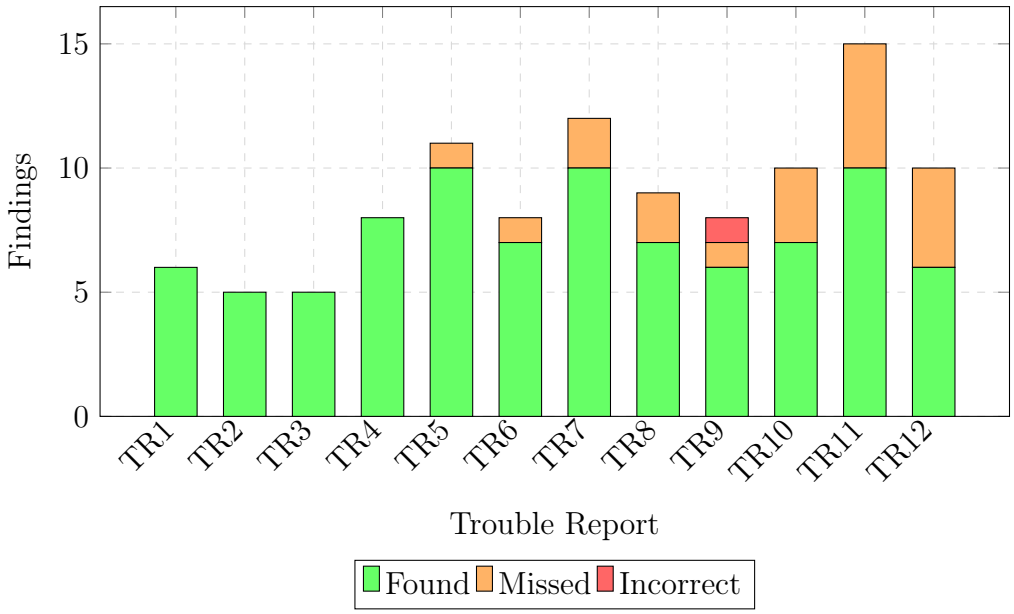


Figure E.2: Case 1: Agent accuracy across 12 trouble reports. Each bar shows the total log-derivable findings, split into found (87), missed (19), and incorrect (1). Overall accuracy: 81%.

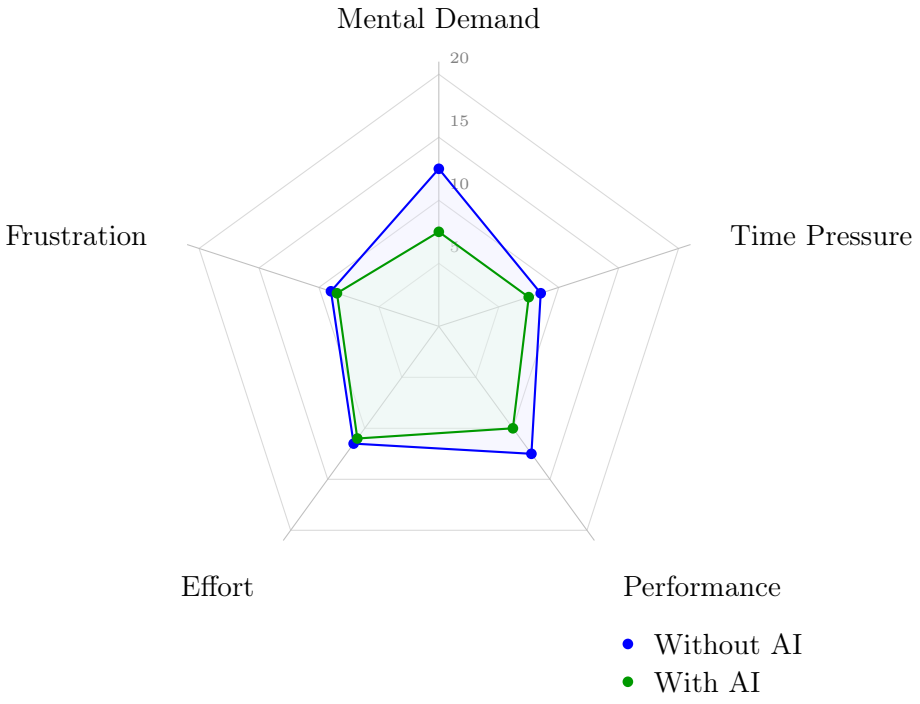


Figure E.3: Case 1: Median NASA-TLX scores across five workload scales (21-point scale). Lower values indicate lower perceived workload.

Case 2: Information Search

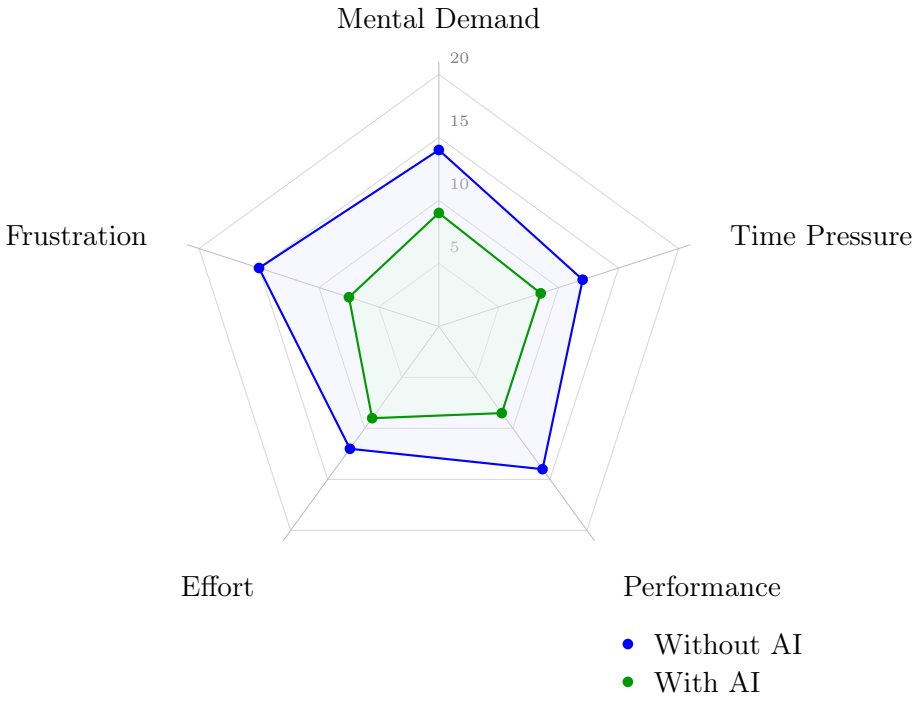


Figure E.4: Case 2: Median NASA-TLX scores across five workload scales (21-point scale). The reduction is larger than in Case 1 across all dimensions.

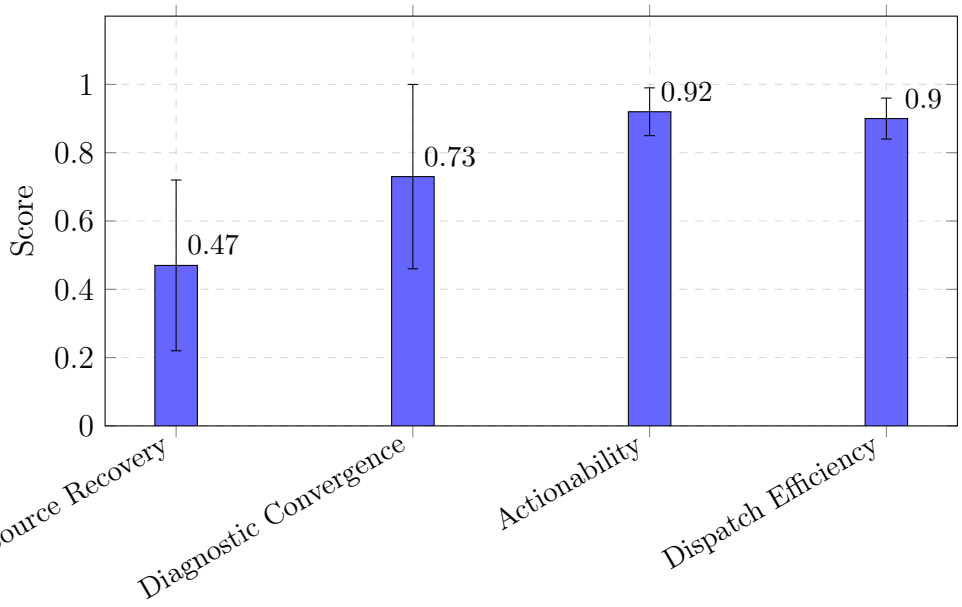


Figure E.5: Case 2: Mean agent accuracy metrics across 90 runs (30 cases $\times$ 3 repeats) with standard deviation error bars. Source Recovery and Diagnostic Convergence show high variance ($\sigma = 0.25$ and $\sigma = 0.27$), while operational metrics are stable.



Faculty of Computing, Blekinge Institute of Technology, 371 79 Karlskrona, Sweden